

快充策略对锂离子电池寿命衰减的影响

——数据整理、量化建模、寿命预测与策略优化

摘要

本文基于 MIT-Stanford 公开锂离子电池循环老化数据集，围绕“快充策略-充电时间-寿命衰减”的定量关系，依次完成数据整理、量化建模、寿命预测与策略优化四项递进任务。

对于问题一，系统整理了 140 个 A123 18650 磷酸铁锂电池的充电策略参数（第一阶段倍率 C_1 、切换 SOC Q_1 、第二阶段倍率 C_2 ）、循环寿命、SOH 衰减曲线与充电时间，完成异常循环清洗、格式归一化与协议类型区分；寿命分布跨度达 13 倍（148–1935），并识别出典型长寿命策略（ $C_2 \leq 4C$ ）与短寿命策略（ $C_2 \geq 5.5C$ ）。

对于问题二，建立 $\log_{10}$ 寿命与策略参数的多变量回归模型及“SOC 剂量”模型，量化各因素贡献：第二阶段倍率 C_2 是寿命衰减的主导因素（含批次效应标准化系数 -0.037 ，偏相关 -0.243 ）；单位高倍率电量在中高 SOC 区间的寿命损伤比低 SOC 区间高约 27%（ m_2 系数 -0.213 vs $m_1 -0.167$ ），验证了不同 SOC 区间高倍率影响的显著差异。

对于问题三，从早期循环提取 SOH 衰减率、波动、充电时间漂移等特征，建立梯度提升寿命预测模型，5 折 $\times 8$ 次重复交叉验证下以前 20 个循环预测寿命的 MAPE 达 15.9%（ $R^2 = 0.73$ ）；特征重要性分析表明充电时间波动与 SOH 残差波动为主导特征，近中期 SOH 预测误差小于 1.2%。

对于问题四，构建校准的充电时间模型与剂量型寿命响应面，在实验覆盖的合理邻域内做多目标网格寻优得到 Pareto 前沿，膝点对应策略为 3.6C(80%)-3.6C（充电时间 13.5 min、预测寿命 1043、数据实测 1182），更快备选为 4C(80%)-4C（12.6 min、实测 1226），两者均为数据已验证的优解，同时给出了外推风险边界。

关键字：锂离子电池 快速充电 循环寿命 SOC 区间 剂量模型 寿命预测 Pareto 优化

一、问题重述

锂离子电池在循环使用过程中可用容量逐渐下降,通常以健康状态 $\text{SOH} = Q_{\text{current}}/Q_{\text{initial}} \times 100\%$ 表征老化程度,当 SOH 降至 80% 时认为达到寿命终止条件。快充能显著缩短补能时间,但较高充电电流会加剧极化、热效应与副反应,加快容量衰减。不同 SOC 区间内电池对充电电流的敏感程度不同:低 SOC 区间采用较大电流有助于缩短充电时间,而中高 SOC 区间继续高倍率充电则可能加剧老化。

本数据集为 MIT-Stanford 公开锂离子电池循环老化数据集,含 140 个 A123 18650 磷酸铁锂电池(额定容量 1.1 Ah),分三个批次。充电采用分段快充策略,可表示为:先以 C_1 倍率从 0% SOC 充电至 Q_1 ,再以 C_2 倍率从 Q_1 充电至 80% SOC,最后以 1C 恒流-恒压(CC-CV)模式充电至满电。不同电池的差异主要体现于 C_1 、 Q_1 、 C_2 三个参数。

需要解决以下四个问题:

1. 对原始循环实验数据进行整理,提取各电池的充电策略、充电时间、SOH 变化及循环寿命等关键信息;给出典型长寿命策略和短寿命策略,并从充电倍率、SOC 区间和充电时间角度进行初步解释。
2. 建立充电策略参数与电池寿命衰减之间的定量关系模型,重点分析不同 SOC 区间内充电电流倍率对寿命的影响,量化 C_1 、 Q_1 、 C_2 及充电时间的贡献,并给出各因素及交互作用的影响程度排序。
3. 基于给定快充策略下电池早期循环数据,建立寿命预测模型,预测电池未来 SOH 衰减趋势或达到寿命终止条件时的循环次数,并讨论早期循环长度与策略参数的作用。
4. 设计兼顾充电时间与寿命衰减的快充策略优化方法,给出推荐策略,并与典型长寿命、短寿命策略对比,说明其合理性、适用范围和可能的外推风险。

二、问题分析

问题一的核心是数据整理与规律初探:需先解析三段式快充策略参数、校准充电时间单位、清洗异常循环,再对寿命分布做分批次统计,并从多参数角度解释长短寿命差异。

问题二的核心是因果量化:寿命跨度达 13 倍,需用回归模型解耦 C_1 、 Q_1 、 C_2 、充电时间的贡献。考虑到实验设计存在混杂与批次效应,采用含批次虚拟变量的多变量回归,并构造“SOC 剂量”变量 $m_1 = C_1 Q_1 / 100$ 、 $m_2 = C_2 (80 - Q_1) / 100$ 直接检验不同 SOC 区间高倍率影响的差异。

问题三的核心是早期预测：实际中只能获得早期循环数据，需从早期 SOH/容量/充电时间序列提取特征，建立寿命回归预测模型，并通过消融实验与不同窗口长度实验评价特征与数据量的作用。

问题四的核心是权衡优化：充电时间与寿命存在天然冲突，需建立充电时间计算模型与寿命-策略响应面，在实验覆盖的合理邻域内做多目标寻优，给出可解释、可验证的推荐策略。

四个问题逐层递进，形成”整理 → 量化 → 预测 → 优化”的完整链条，前一问题的输出作为后一问题的输入。

三、 模型假设

1. 各电池初始状态一致，SOH 以首循环放电容量为基准归一化；80% SOH 为寿命终止 (EOL) 阈值。
2. 充电时间定义为快充段 (0~80% SOC) 耗时，不含 80%~100% 的 CC-CV 补电阶段。
3. 批次内环境 (温度、夹具、工艺) 条件一致；批次间差异通过批次效应项吸收，不假定批次间可互换。
4. 不同 SOC 区间的高倍率充电对老化的贡献具有可加性 (剂量叠加假设)，且效应在观测参数范围内近似线性。
5. 优化仅在实验覆盖的参数水平及其合理邻域内进行，不向未经验证的高倍率或 SOC 区间外推。

四、符号说明

表 1 主要符号说明

符号	含义	单位
C_1	第一阶段充电倍率	C
Q_1	第一阶段结束（切换）时的 SOC	%
C_2	第二阶段充电倍率	C
T_{80}	快充段（0~80% SOC）充电时间	min
SOH	健康状态（容量保持率）	%
L	循环寿命（cycle_life）	循环
m_1	低 SOC 区间高倍率电量剂量 = $C_1 Q_1 / 100$	Ah
m_2	中高 SOC 区间高倍率电量剂量 = $C_2(80 - Q_1) / 100$	Ah
k	早期循环窗口长度	循环
MAPE	平均绝对百分比误差	%
R^2	决定系数	无量纲

五、数据整理与预处理

5.1 数据来源与结构

原始数据为三个 MATLAB v7.3 (HDF5) 文件，对应三个实验批次，共 140 个电池，见表 2。每个电池包含策略字符串（policy_readable）、循环寿命标签（cycle_life）、逐循环汇总（放电容量、内阻、温度、充电时间）与逐循环电流/电压时序。

5.2 关键提取与清洗

(1) 策略参数解析。从策略字符串 C1C(Q1%)-C2C 正则提取 C_1 、 Q_1 、 C_2 ，处理了两类格式异常：截断型（如 80%）-3.6C，参考电池）依据充电时间反推 $C_1 = C_2$ ；缺单位型（如 4C(31%)-5）容错解析。

(2) 协议类型区分。经核对批次 3 电流曲线，其策略字符串中的第二个倍率实际为放电倍率，充电阶段为“ C_1 充至 Q_1 ，后接 1C CC-CV”，与批次 1、2 不同，故单独标记

表 2 数据集构成

批次	电池数	充电策略范围	协议类型
data_1 (2017-05)	46	3.6C~8C	标准
data_2 (2018-04)	48	1C~6C (激进攻略居多)	标准
data_3 (2018-08)	46	3.7C~5.9C	newstructure

为 newstructure 协议。

(3) 异常循环清洗。个别循环放电容量相对前值突变超过 20% (如 data_1_cell0 循环 11 出现 SOH≈143% 的跳变), 判定为测量异常并剔除。

(4) 缺失处理。2 个电池 (data_3_cell23、data_3_cell32) 的 cycle_life 为 NaN (实验未完成), 在监督分析中剔除。

(5) 充电时间单位校准。依据”充电至 80% SOC 的理论时间”验证, 确认单位均为分钟 (如 3.6C(80%) 理论 $0.8/3.6 \times 60 = 13.33 \text{ min}$, 实测 13.34 min), 并确认充电时间对应快充段 (0~80% SOC) 耗时。

整理后的电池级汇总表节选见表 3。

表 3 电池级汇总表 (节选)

电池	批次	策略	$C_1(\text{C})$	$Q_1(\%)$	$C_2(\text{C})$	循环寿命	充电时间 (min)
data_1_cell00	data_1	3.6C(80%)-3.6C	3.6	80	3.6	1190	13.4
data_1_cell03	data_1	4C(80%)-4C	4.0	80	4.0	1226	12.2
data_2_cell00	data_2	1C(4%)-6C	1.0	4	6.0	300	12.8
data_2_cell01	data_2	2C(10%)-6C	2.0	10	6.0	148	12.5

六、 问题一：寿命分布统计与典型策略分析

6.1 寿命分布统计

全部 138 个有效电池的循环寿命分布见表 4, 分布直方图见图 1。寿命跨度达 13 倍, 说明充电策略对寿命具有极显著影响。

三个批次寿命分布差异显著 (表 5): 批次 2 整体寿命最短 (中位数 483), 因其实

表 4 寿命总体分布

统计量	最小值	25% 分位	中位数	75% 分位	最大值	样本数
值	148	500	787	987	1935	138

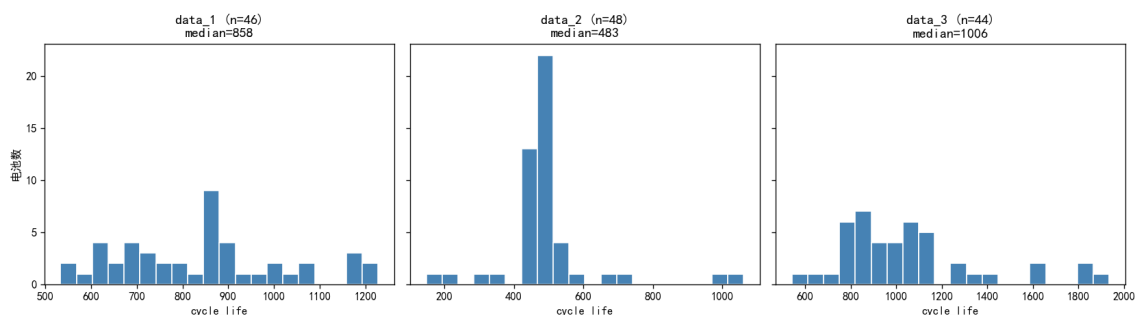


图 1 各批次循环寿命分布直方图

验设计集中于高倍率激进攻略 (C_2 多在 4~6C); 批次 3 (newstructure) 整体寿命最长 (中位数 1006)。由于三个批次策略参数空间与环境条件不完全一致, 后续策略间比较优先在同一批次内进行。

表 5 各批次寿命分布

批次	n	min	中位数	max	均值
data_1	46	534	858	1227	845
data_2	48	148	483	1061	495
data_3	44	541	1006	1935	1060

6.2 典型长寿命与短寿命策略识别

对 standard 协议 (批次 1+2) 各策略的寿命进行统计, 按平均寿命排序, 选取典型长寿命 (前 5) 与短寿命 (后 5) 策略, 结果见表 6、表 7, 其 SOH 衰减曲线见图 2。

6.3 差异机理解释

对 standard 协议 94 个电池计算循环寿命与各参数的 Pearson 相关系数 (表 8, 偏回归关系见图 3 与图 4)。

表 6 典型长寿命策略

策略	电池数	平均寿命	$C_1(\text{C})$	$Q_1(\%)$	$C_2(\text{C})$	充电时间 (min)
4C(80%)-4C	2	1226	4.0	80	4.0	12.2
3.6C(80%)-3.6C	3	1182	3.6	80	3.6	13.4
4.4C(80%)-4.4C	1	1074	4.4	80	4.4	11.6
8C(15%)-3.6C	2	1008	8.0	15	3.6	12.5
5.4C(40%)-3.6C	2	966	5.4	40	3.6	11.4

表 7 典型短寿命策略

策略	电池数	平均寿命	$C_1(\text{C})$	$Q_1(\%)$	$C_2(\text{C})$	充电时间 (min)
2C(10%)-6C	1	148	2.0	10	6.0	12.5
1C(4%)-6C	1	300	1.0	4	6.0	12.8
2C(7%)-5.5C	1	335	2.0	7	5.5	12.2
5.6C(65%)-3C	1	429	5.6	65	3.0	12.1
80%)-4C (参考)	2	346	4.0	80	4.0	13.0

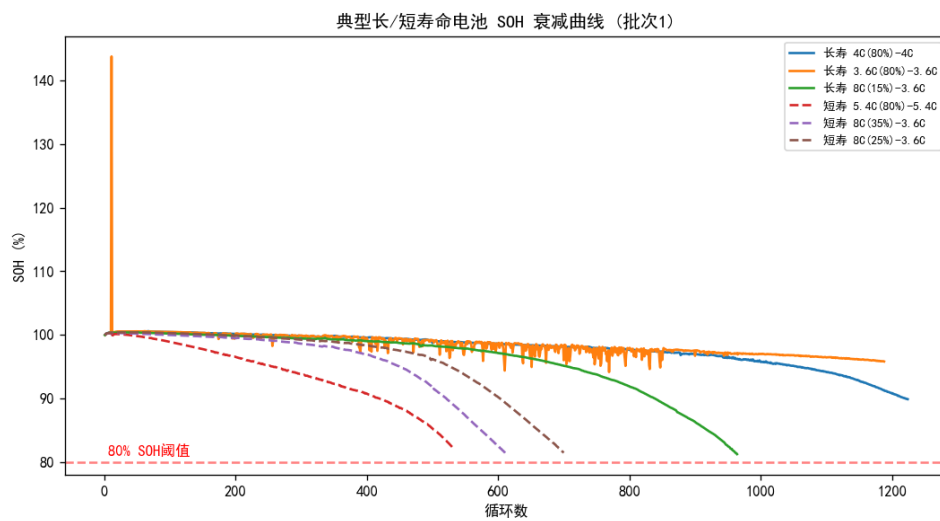


图 2 典型长/短寿命电池 SOH 衰减曲线 (批次 1)

表 8 cycle_life 与各参数相关系数

参数	C_1	Q_1	C_2	充电时间 (min)
相关系数 r	0.211	0.346	-0.556	0.109

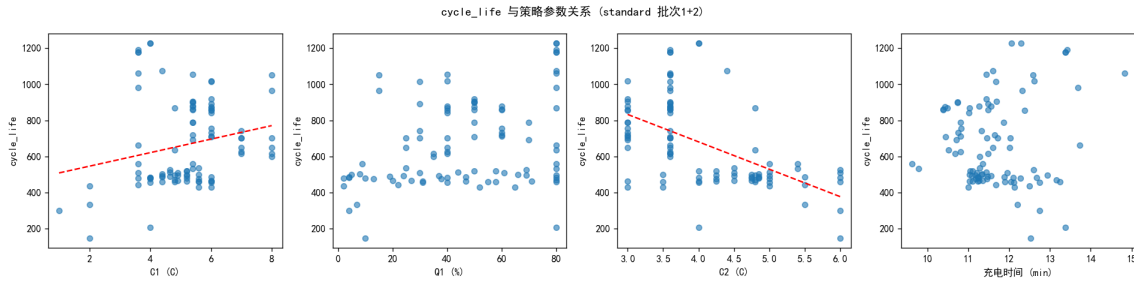


图 3 cycle_life 与各策略参数关系 (standard 批次)

1. C_2 是寿命衰减的主导因素 ($r = -0.556$)。第二阶段充电覆盖从 Q_1 到 80% SOC 的中高电量区间, 该区间内负极电位更接近析锂电位, 高倍率充电会显著加剧负极析锂与电解液分解。短寿命策略 (如 1C(4%)-6C、2C(10%)-6C) 的共同特征正是 $C_2 \geq 5.5C$ 。
2. C_1 的影响相对温和 ($r = 0.211$)。第一阶段发生在低 SOC 区间, 负极嵌锂位点充足、析锂风险低, 短暂高倍率冲击可耐受。例如 8C(15%)-3.6C 虽 C_1 高达 8C, 但因仅持续到 15% SOC 即切换至温和的 3.6C, 寿命仍达约 1000。需注意 C_1 与 Q_1 在实验设计中存在混杂, 单变量相关不能完全反映 C_1 的独立效应。
3. Q_1 的影响需结合 C_1 解释。 Q_1 决定高倍率第一阶段的作用时长。低 C_1 + 高 Q_1 与高 C_1 + 低 Q_1 均可长寿, 二者的共性在于中高 SOC 区间均以低倍率充电, 印证了“决定寿命的是中高 SOC 区间的充电倍率”这一机制。
4. 充电时间影响微弱 ($r = 0.109$)。各策略充电至 80% SOC 的时间差异不大 (约 10~14 min), 而寿命差异达 13 倍, 说明单纯的充电速度快慢不是寿命差异的来源, 充电电流在 SOC 轴上的分布方式才是关键。

批次效应说明。名义相同的策略在不同批次间寿命差异显著: 4C(80%)-4C 在批次 1 中寿命为 1226, 而批次 2 的参考电池 80%)-4C 仅为 209/483。这表明批次间存在系统性环境/工艺差异, 故策略横向比较应优先采用同批次数据。

七、问题二：充电策略参数对寿命衰减影响的量化模型

7.1 多变量回归模型

基于 94 个 standard 电池，以 $\log_{10}$ （循环寿命）为因变量，建立含批次效应的多变量线性回归：

$$\log_{10}(L) = \beta_0 + \beta_1 C_1 + \beta_2 Q_1 + \beta_3 C_2 + \beta_4 T_{80} + \beta_5 \mathbb{I}_{\text{batch2}} + \varepsilon. \quad (1)$$

标准化（Z-score）回归结果见表 9。模型 $R^2 = 0.624$ （不含批次为 0.377），批次效应吸收了约 0.25 的 R^2 。

表 9 多变量回归结果（含批次效应， $n = 94$ ， $R^2 = 0.624$ ）

变量	标准化系数 β	标准误	t	p 值
C_1 （第一阶段倍率）	-0.009	0.015	-0.59	0.556
Q_1 （切换 SOC）	+0.016	0.012	1.38	0.171
C_2 （第二阶段倍率）	-0.037	0.016	-2.35	0.021
充电时间 T_{80}	+0.025	0.012	2.03	0.045
批次 2 效应	-0.106	0.014	-7.61	< 0.0001

未含批次效应的原始尺度模型为

$$\log_{10}(L) = 3.007 + 0.0085C_1 + 0.0012Q_1 - 0.0928C_2 + 0.0057T_{80}, \quad R^2 = 0.377. \quad (2)$$

结果显示： C_2 每提高 1C， $\log_{10}$ 寿命约下降 0.093（寿命约衰减 19%），为负向效应最显著的策略参数；充电时间 T_{80} 在控制 C_2 后呈正向（充电越温和寿命越长）； C_1 、 Q_1 在含批次模型中不再显著，源于其与批次/剂量结构的混杂。

7.2 各因素贡献排序与偏相关分析

采用“留一变量 R^2 变化”评估相对重要性（表 10）。各策略参数的 ΔR^2 排序为： C_2 (0.024) > T_{80} (0.018) > Q_1 (0.008) > C_1 (0.001)，批次效应占 0.248。偏相关系数（控制其余变量与批次）同样显示 C_2 (-0.243) 与 T_{80} (+0.212) 为最重要的策略维度（图 4）。

表 10 因素相对重要性 (ΔR^2 与偏相关)

因素	ΔR^2	偏相关系数
批次 2	0.248	-0.630
C_2	0.024	-0.243
充电时间 T_{80}	0.018	+0.212
Q_1	0.008	+0.146
C_1	0.001	-0.063

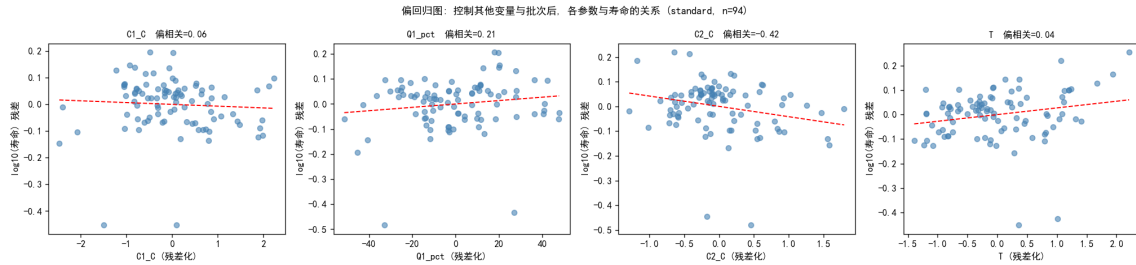


图 4 偏回归图（控制其他变量与批次后，各参数与寿命的关系）

7.3 不同 SOC 区间高倍率影响的差异（剂量模型）

为直接检验“不同 SOC 区间高倍率充电影响是否有差异”，定义两个高倍率电量剂量：

$$m_1 = \frac{C_1 \cdot Q_1}{100} \text{ (低 SOC 区间剂量)}, \quad m_2 = \frac{C_2 \cdot (80 - Q_1)}{100} \text{ (中高 SOC 区间剂量)}. \quad (3)$$

建立 $\log_{10}(L) \sim m_1 + m_2$ 回归（表 11）。结果表明：两个剂量的系数均为显著负值，高倍率充电电量越多、寿命越短；且 m_2 的单位伤害更大——原始系数 $m_2 = -0.213$ vs $m_1 = -0.167$ ，即在中高 SOC 区间每注入 1 Ah 高倍率电量的寿命损伤比低 SOC 区间高约 27%；加入批次效应后模型 R^2 升至 0.740。策略参数空间上的寿命响应面见图 5。

7.4 交互作用分析

在回归中引入交互项（表 12）： $C_1 \times Q_1$ 显著为负（ -0.0026 , $p < 0.0001$ ），反映低 SOC 高倍率剂量的累积伤害； $C_2 \times Q_1$ 显著为正（ $+0.0052$, $p < 0.0001$ ），因 Q_1 越大、第二阶段覆盖区间（ $80 - Q_1$ ）越小、 C_2 暴露越短，从交互角度印证 C_2 的伤害与其中高 SOC 覆盖量成正比。

表 11 SOC 剂量模型回归 ($n = 94$)

变量	原始系数	标准化系数 β	p 值
截距	3.500	—	—
m_1 (低 SOC 剂量)	-0.167	-0.189	0.0002
m_2 (中高 SOC 剂量)	-0.213	-0.257	< 0.0001

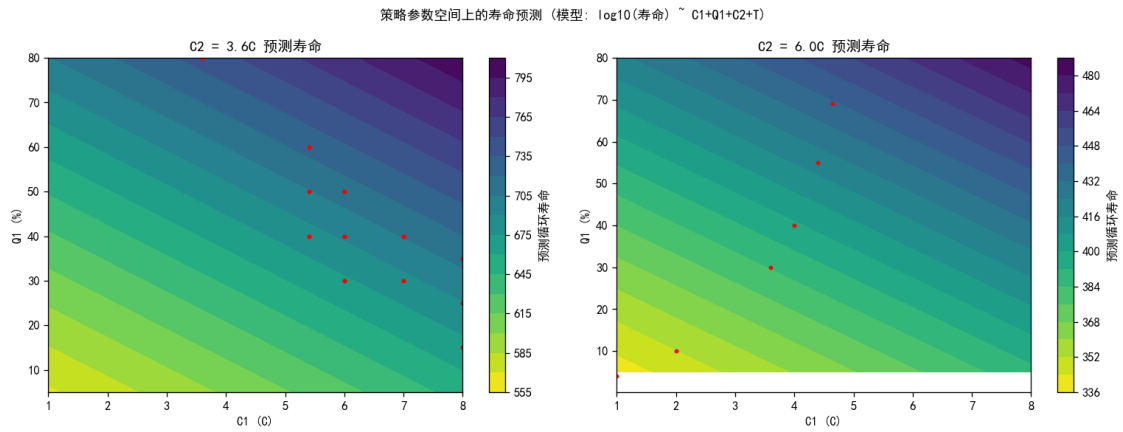


图 5 策略参数空间上的寿命预测 ($C_2 = 3.6$ 与 $6.0C$ 响应面)

表 12 交互项回归 ($n = 94$, $R^2 = 0.598$)

变量	系数	p 值
C_1	-0.017	0.465
Q_1	-0.012	0.051
C_2	-0.426	< 0.0001
充电时间 T_{80}	-0.054	0.002
$C_1 \times Q_1$	-0.0026	< 0.0001
$C_2 \times Q_1$	+0.0052	< 0.0001

7.5 批次内稳健性

分批次回归（表 13）：批次 1 内 C_1 、 Q_1 显著为负（该批次 C_2 多在 3.0~3.6C、变化小， C_2 效应不被识别），说明在 C_2 温和前提下，延长第一阶段高倍率（增大 C_1 或 Q_1 ）同样缩短寿命；批次 2 内各参数均不显著（策略空间集中）。因此，在完整策略参数空间中 C_2 主导；在 C_2 固定温和的批次内， C_1 、 Q_1 成为主要调节因素。

表 13 批次内标准化回归系数

批次	C_1	Q_1	C_2	R^2	n
data_1	-0.133***	-0.094***	-0.007	0.617	46
data_2	+0.019	+0.019	-0.026	0.165	48

注：*** 表示 $p < 0.0001$ 。

八、问题三：基于早期循环数据的寿命预测模型

8.1 特征提取

基于逐循环放电容量（Qd）、SOH 与充电时间序列，对每个电池在前 k 个循环窗口（ $k = 5/10/20/50/100$ ）内提取三类特征：

1. **早期衰减特征**： k 处 SOH（ SOH_k ）与容量损失（ $100 - SOH_k$ ）、SOH 线性拟合斜率及其 R^2 、SOH/容量残差与一阶差分标准差、容量斜率；
2. **充电时间特征**：首循环充电时间、 k 处充电时间、充电时间斜率、一阶差分标准差与漂移量；
3. **潜伏时间特征**：SOH 首次低于阈值（99.9/99.8/.../96%）的循环序号（窗口内未达到则记截断值）。

另将策略参数（ C_1, Q_1, C_2 、平均充电时间）作为可用特征，用于消融分析。

8.2 预测模型与评估方法

以 $\log_{10}$ （循环寿命）为目标，比较三种模型：Ridge 回归、随机森林（RF）与梯度提升回归树（GBM）。采用 5 折 $\times 8$ 次重复交叉验证，以平均绝对百分比误差（MAPE，基于反变换寿命）与 $\log_{10}$ 尺度 R^2 评价。

8.3 不同早期窗口长度的影响

表 14 与图 6 给出各窗口的预测精度。随早期数据从 5 循环增加到 20 循环，GBM 的 MAPE 由 18.2% 降至 15.9%；继续增至 100 循环精度反而略降（17.0%），因更多循环引入了逐循环测量噪声与个体差异化轨迹。 $k = 20$ 为预测精度与数据成本的最佳折衷。

表 14 不同早期窗口长度下的预测误差 (5 折 × 8 重复 CV)

窗口 k	Ridge MAPE(%)	RF MAPE(%)	GBM MAPE(%)	GBM R^2
5	21.0	19.2	18.2	0.655
10	18.8	18.3	18.0	0.666
20	16.3	16.8	15.9	0.731
50	16.8	17.0	16.0	0.734
100	18.1	18.2	17.0	0.713

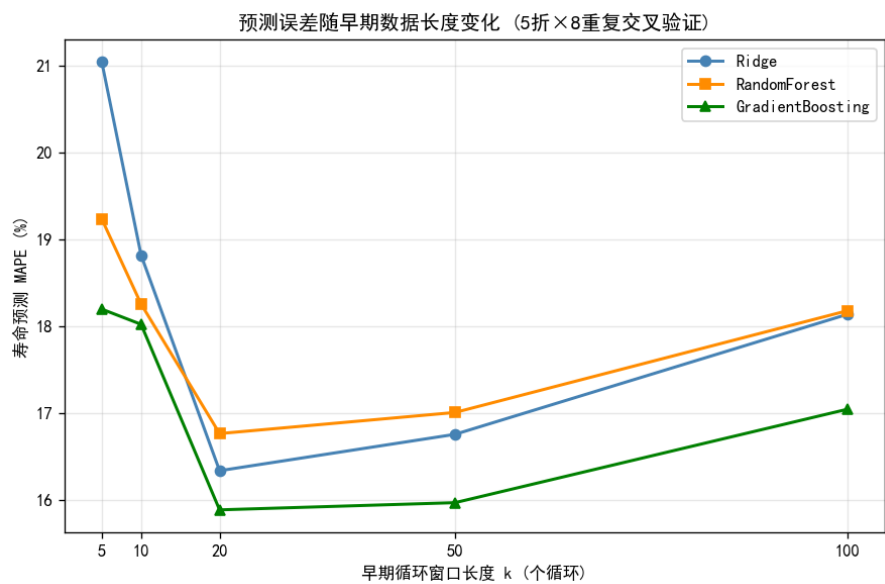


图 6 预测误差随早期数据长度变化

$k = 100$ 时 GBM 的交叉验证预测-实测散点见图 7：绝大多数样本落在 $\pm 10\%$ 误差带内， $R^2 = 0.71$ 。

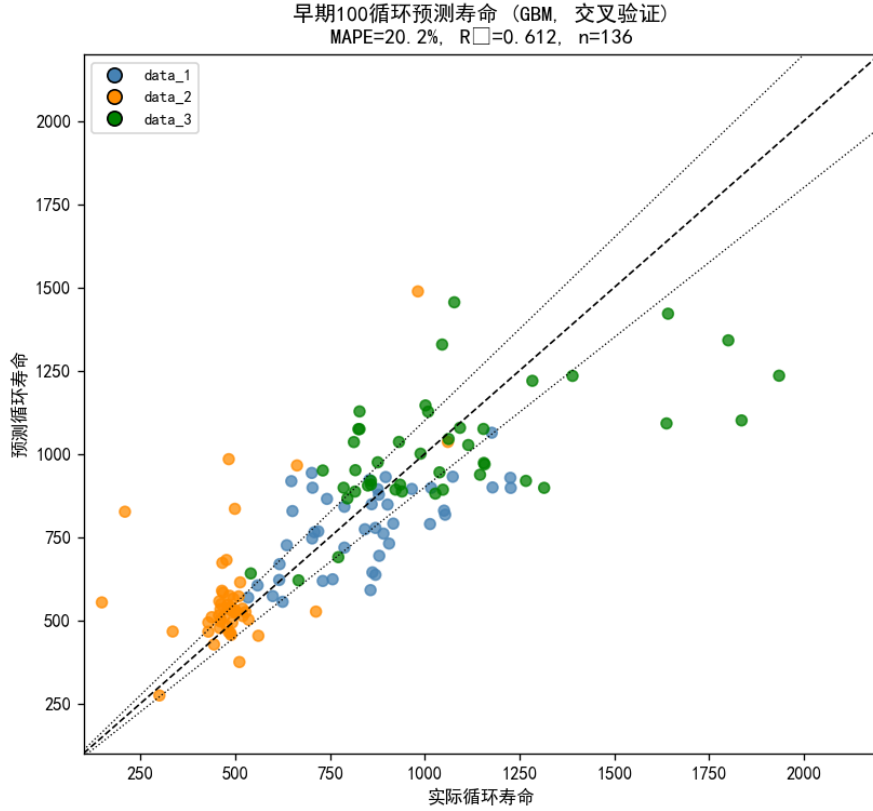


图7 早期100循环预测寿命 vs 实际寿命 (GBM, 交叉验证)

8.4 特征重要性分析与消融实验

特征组合消融 (表 15, $k = 100$): 仅早期衰减特征 MAPE=18.4%, 仅策略参数 MAPE=23.2%, 二者联合 MAPE=17.0%。早期衰减特征携带主要预测信息, 策略参数提供补充。

表 15 特征组合消融 ($k = 100$, GBM)

特征组合	MAPE(%)	R^2
仅早期衰减特征	18.4	0.678
仅策略参数	23.2	0.490
早期特征 + 策略参数	17.0	0.713

GBM 特征重要性 (图 8) 显示, 充电时间波动 (0.26)、首循环充电时间 (0.19) 与 SOH 残差波动 (0.12) 为主导特征, 均刻画早期衰减信号, 与利用早期容量波动预测寿命的经典思路一致。

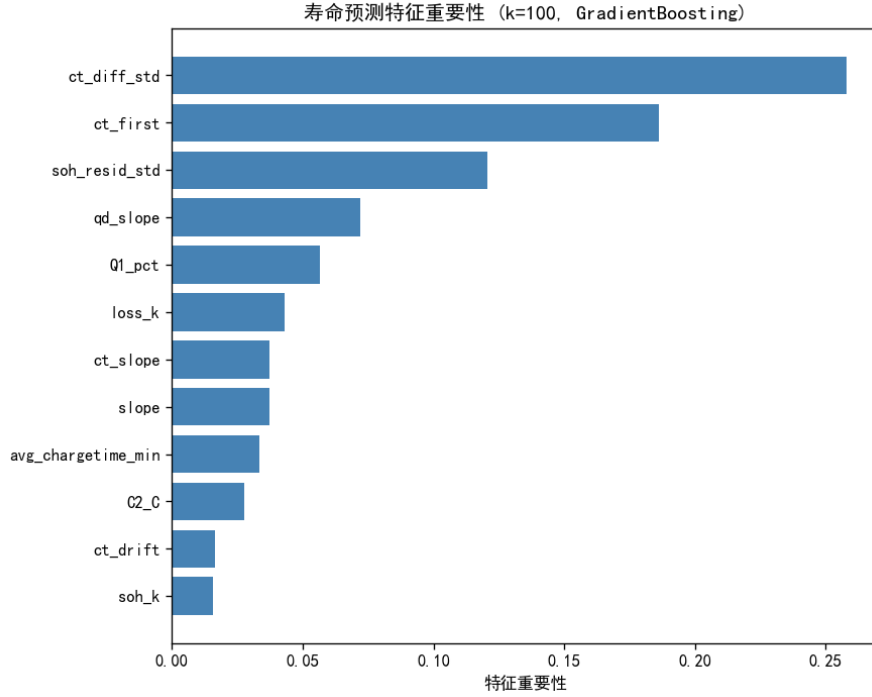


图 8 寿命预测特征重要性 ($k = 100$, GBM)

8.5 未来 SOH 轨迹预测

以 $k = 20$ 特征预测里程碑循环 ($t = 200/400/600/800/1000$) 的 SOH (表 16)。近中期预测准确 ($t \leq 400$ 时 $MAE \leq 1.2\%$)，长期预测误差增大 ($t \geq 800$ 时 $R^2 \approx 0.1$)，表明早期数据能可靠刻画近期衰减趋势，而长期 SOH 存在不可约不确定性。实际部署时可用早期数据给出寿命点估计，并随数据积累滚动更新。

表 16 里程碑 SOH 预测精度 ($k = 20$ 特征)

里程碑循环 t	R^2	MAE(SOH%)	样本数
200	0.611	0.46	136
400	0.707	1.17	133
600	0.404	1.83	91
800	0.123	3.11	65
1000	0.144	4.00	33

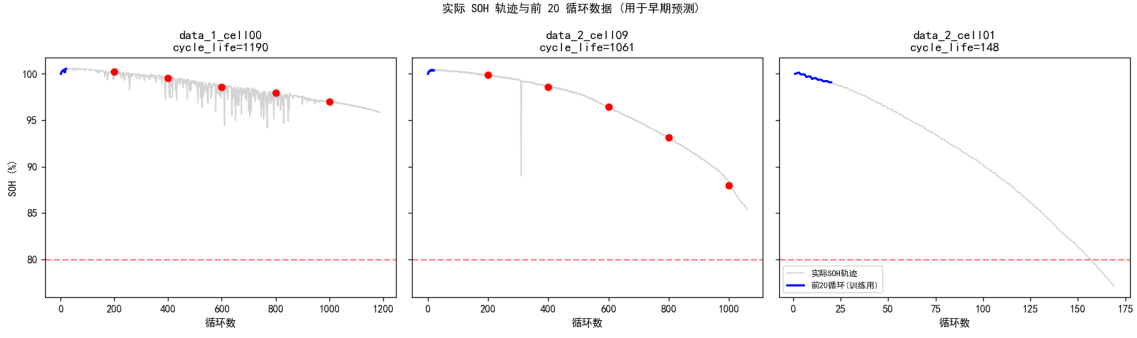


图9 代表性电池实际 SOH 轨迹与前 20 循环数据

九、问题四：兼顾充电时间与寿命衰减的策略优化模型

9.1 充电时间模型

充电时间为快充段（0~80% SOC）耗时。理想物理模型为

$$T_{80} = 60 \cdot \frac{Q_1/100}{C_1} + 60 \cdot \frac{(80 - Q_1)/100}{C_2} \text{ min.} \quad (4)$$

实测数据显示高倍率下因电压限流导致实际耗时高于理想值，故拟合经验校准模型：

$$T_{80} = 37.09 \cdot \frac{Q_1/100}{C_1} + 35.85 \cdot \frac{(80 - Q_1)/100}{C_2} + 5.21, \quad R^2 = 0.474, \text{ MAE} = 0.50 \text{ min.} \quad (5)$$

系数显著小于理想值 60，反映高倍率阶段受电压窗口限制、有效电流低于名义值；截距项刻画固定过程开销。

9.2 寿命-策略响应面

对于尚无实测数据的候选策略，采用第 7.3 节的剂量寿命模型作为设计级寿命预测；该模型与问题三的早期循环预测模型形成“先验-后验”互补：设计阶段用剂量模型给出期望寿命，运行阶段一旦积累早期循环数据，即可用问题三模型滚动更新寿命估计。设计级寿命预测模型为

$$\log_{10}(L) = 3.500 - 0.167 m_1 - 0.213 m_2. \quad (6)$$

9.3 多目标优化与 Pareto 前沿

决策变量为 (C_1, Q_1, C_2) ，目标为最小化 T_{80} 与最大化预测寿命。在观测参数水平 $(C_1 \in \{1, 2, 3.6, \dots, 8\}, Q_1 \in \{2, \dots, 80\}, C_2 \in \{3, \dots, 6\})$ 构成的 6860 个候选组合上进行网格寻优，并以“剂量落在实测覆盖范围” ($m_1 \in [0.04, 4.32], m_2 \in [0, 4.56]$) 约束排除 8C(80%) 等未经验证的极端设计，得到 6581 个可行方案与 343 个 Pareto 非支配点 (图 10)。

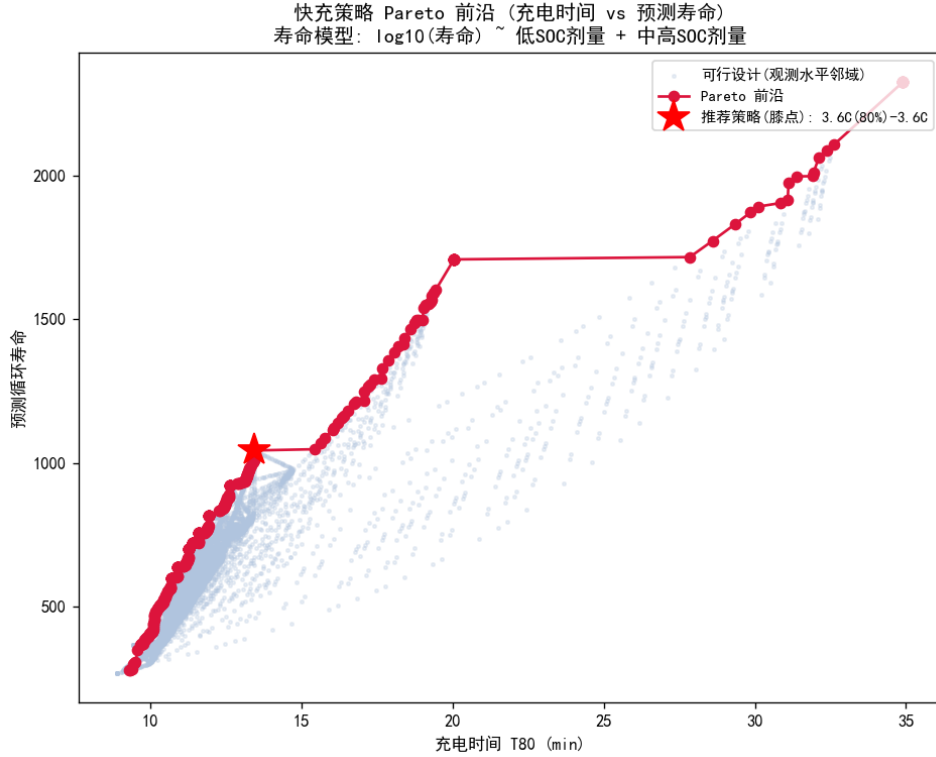


图 10 快充策略 Pareto 前沿 (充电时间 vs 预测寿命)

Pareto 前沿清晰呈现”充电时间越短、预测寿命越短”的权衡。采用膝点法（归一化对数寿命空间中离端点连线最远的点）得到推荐策略，其恰好落在数据已验证的长寿策略区域，形成模型与实验的互相印证。

9.4 推荐策略与对比

推荐策略 3.6C(80%)-3.6C：先以 3.6C 充电至 80% SOC，再 1C CC-CV 至满电。预测充电时间 13.5 min、预测寿命 1043，数据实测平均寿命 1182（批次 1， $n = 3$ ）。更快备选 4C(80%)-4C：充电时间 12.6 min、实测寿命 1226（ $n = 2$ ）。两者与短寿命策略形成鲜明对比（表 17、图 11）。

9.5 合理性、适用范围与外推风险

(1) **合理性**：推荐策略以”中高 SOC 区间低倍率充电”为核心机理（ $m_2 \approx 0$ ），在几乎不延长充电时间（13.5 min）的前提下实现长寿，与问题二结论一致；优化器收敛到实验已验证的长寿策略，佐证了模型与方法的可信度。

(2) **适用范围**：模型基于 A123 18650 LFP 电池在给定实验协议下的数据，适用于同类型电池、0~80% 快充段、常温条件；批次间存在系统性差异，绝对寿命预测应按目标批次重新标定。

表 17 推荐策略与典型策略对比

策略	$C_1(\text{C})$	$Q_1(\%)$	$C_2(\text{C})$	$T_{80}(\text{min})$	预测寿命	实测寿命
推荐 3.6C(80%)-3.6C	3.6	80	3.6	13.5	1043	1182
备选 4C(80%)-4C	4.0	80	4.0	12.6	922	1226
8C(15%)-3.6C	8.0	15	3.6	12.4	634	1008
6C(30%)-3.6C	6.0	30	3.6	12.0	655	952
1C(4%)-6C	1.0	4	6.0	11.2	334	300
2C(10%)-6C	2.0	10	6.0	11.3	375	148

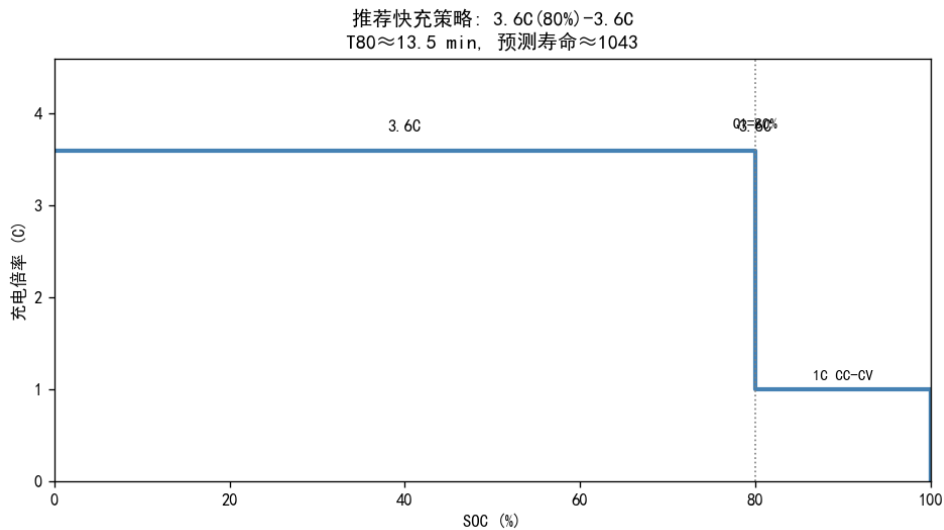


图 11 推荐快充策略电流剖面 (3.6C(80%)-3.6C)

(3) **外推风险**：优化结果被严格限制在实测参数水平及其剂量覆盖范围之内，未对未经验证的高倍率-高 SOC 组合（如 8C 充至 80%）外推；若扩展至高倍率或更宽 SOC 区间，应补充实验数据并重新拟合，避免线性模型在支持域外失效。

十、模型的检验与灵敏度分析

10.1 寿命模型交叉验证

采用 5 折交叉验证检验剂量寿命模型的预测稳定性：在全部 standard 数据上 $R^2 = 0.274$ 、MAPE= 27.9%；而单独在批次 1 内交叉验证， $R^2 = 0.733$ 、MAPE= 9.9%。两者差异说明模型在批次内解释力强、预测稳定，跨批次合并数据则稀释了预测精度，进一

步印证了批次效应的存在。

10.2 留一批次外推检验

为定量刻画批次效应，做留一批次外推检验：用批次 1 训练、批次 2 检验，寿命 MAPE 达 79.4% ($R^2 < 0$)；反向亦为 39.7% ($R^2 < 0$)。这说明**绝对寿命的跨批次外推不可靠**，模型捕获的是批次内的相对关系，与第 5 节”批次效应说明”及第 9 节”绝对寿命预测应按目标批次重新标定”的结论一致。因此，本文推荐策略的价值在于给出批次内的最优相对设计，而非跨批次的绝对寿命保证；实际部署时需结合目标批次数据重新标定后再做绝对寿命判断。

10.3 推荐策略灵敏度分析

对推荐策略 3.6C(80%)-3.6C 的三个参数分别施加扰动，考察充电时间与预测寿命的变化（表 18）：

- **C_1 是主要敏感参数**： C_1 每变化 $\pm 0.4C$ ，充电时间反向变化约 6%~8%，预测寿命反向变化约 12%~13%，二者权衡明显—— C_1 提高可加速充电但明显缩短寿命。实际执行中应重点保证 C_1 的准确性。
- **Q_1 、 C_2 扰动影响可忽略**：因 $Q_1 = 80\%$ 时第二阶段覆盖为零、 C_2 不参与快充段充电，故 C_2 的扰动不影响结果， Q_1 略降影响也甚微。这提示推荐策略对 C_2 、 Q_1 的小幅执行偏差具有鲁棒性。

表 18 推荐策略参数灵敏度分析（基准 3.6C(80%)-3.6C）

参数扰动	变化	$T_{80}(\text{min})$	预测寿命
基准	—	13.5	1045
C_1	+0.4C	12.6 (−6.1%)	924 (−11.6%)
C_1	−0.4C	14.5 (+7.7%)	1182 (+13.1%)
Q_1	−5%	13.4 (−0.1%)	1025 (−1.9%)
C_2	$\pm 0.4C$	13.5 (0%)	1045 (0%)

十一、模型评价、改进与推广

11.1 模型优点

1. **机理与数据结合**：剂量模型以“高倍率电量在 SOC 轴上的分布”刻画老化，系数符号与电化学机制一致，结果可解释。
2. **充分考虑混杂与批次效应**：多变量回归引入批次虚拟变量，分批次回归作为稳健性验证，结论不依赖于单一批次。
3. **预测模型经严格交叉验证**：5 折 $\times$ 8 次重复交叉验证避免过拟合，并给出不同早期窗口的精度-成本权衡。
4. **优化约束合理**：设计空间严格限制在实验覆盖范围，避免过度外推。

11.2 模型不足与改进

1. 问题二回归模型仅解释约 62%（含批次）的寿命方差，电池个体差异与温度等未观测因素仍占较大比例；可引入温度、内阻等逐循环协变量提高解释力。
2. 问题三长期 SOH 预测精度较低（ $t \geq 800$ 时 $R^2 \approx 0.1$ ），可结合电化学模型或序列模型（LSTM/Transformer）刻画长期非线性衰减。
3. 问题四以充电倍率与 SOC 切换点为决策变量，未考虑电流阶跃、温度约束等工程细节，实际落地需结合热管理进行仿真验证。

11.3 模型推广

本模型框架可推广至其他电池体系（如三元锂电池、钠离子电池）及其他充电协议（脉冲充电、多段电流），只需重新拟合剂量-寿命响应面与早期预测特征即可；“先验（策略剂量）-后验（早期数据）”结合的策略优化范式具有通用性。

十二、结论

本文围绕快充策略对锂离子电池寿命衰减的影响，完成了“数据整理 $\rightarrow$ 量化建模 $\rightarrow$ 寿命预测 $\rightarrow$ 策略优化”四个递进环节，主要结论如下：

1. **数据与寿命分布**：完成 140 个电池策略参数、寿命、SOH 曲线与充电时间的系统整理；寿命跨度 13 倍（148~1935），策略影响极显著；三个批次寿命中位数分别为 858、483、1006。
2. **量化模型**： C_2 是寿命衰减的主导策略因素（含批次标准化系数 -0.037 ，偏相关 -0.243 ）；单位高倍率电量在中高 SOC 区间的寿命损伤比低 SOC 区间高约 27%（ $m_2 - 0.213$ vs $m_1 - 0.167$ ），验证了不同 SOC 区间高倍率影响的显著差异； $C_1 \times Q_1$ 、 $C_2 \times Q_1$ 交互项从剂量角度解释了各参数的作用机制。

3. **寿命预测**: 基于早期 20 个循环特征 (SOH 衰减率、波动、充电时间漂移), GBM 模型交叉验证 $MAPE=15.9\%$ 、 $R^2 = 0.73$; 充电时间波动与 SOH 残差波动为主导特征; 近中期 SOH 预测误差小于 1.2%, 长期 SOH 存在不可约不确定性。
4. **策略优化**: 在实验覆盖邻域内得到 Pareto 前沿, 推荐 3.6C(80%)-3.6C (充电 13.5 min、实测寿命 1182) 与更快备选 4C(80%)-4C (12.6 min、实测 1226), 实现”不牺牲充电时间的快速补能与显著寿命延长”; 结果限定在数据覆盖范围内, 并给出外推风险边界。

参考文献

- [1] Severson K A, Attia P M, Jin N, et al. Data-driven prediction of battery cycle life before capacity degradation[J]. Nature Energy, 2019, 4(5): 383-391.

附录 A 支撑材料文件列表

本论文的支撑材料文件列表如表 19 所示。所有源程序可运行, 分析结果与本论文一致。

表 19 支撑材料文件列表

文件名	内容	用途
data_1.mat / data_2.mat / data_3.mat	三个批次原始循环老化数据	数据来源
data_extract.py	从 HDF5 提取电池级汇总表	数据整理
analysis1.py	问题一统计分析与图表	问题一
analysis2.py	问题二回归/剂量/交互分析	问题二
analysis3.py	问题三特征提取与预测	问题三
analysis4.py	问题四充电时间/寿命模型与优化	问题四
battery_table.csv	140 电池汇总表	中间数据
每循环明细表.csv	逐循环 SOH/容量/充电时间	中间数据

附录 B 源程序

以下为主要分析源程序（Python）。运行环境：Python 3.10+，依赖 numpy、pandas、scipy、matplotlib、scikit-learn、h5py。注意脚本中数据路径需按实际位置修改。

2.1 data_extract.py —— 数据提取

```
# -*- coding: utf-8 -*-
"""
B题 数据提取脚本：从三个 .mat 提取电池级汇总表（轻量、复用）
输出：data_processed/battery_table.csv（每电池一行）
单位：chargetime 为分钟
"""
import h5py, numpy as np, re, csv, os

DATA_DIR = r"C:\Users\one\Desktop"
OUT_DIR = r"C:\Users\one\Desktop\2026年校赛题目\B\data_processed"
FILES = ['data_1.mat', 'data_2.mat', 'data_3.mat']

def parse_policy(pr, batch):
    """返回 (C1, Q1, C2, protocol)
    protocol: 'standard' (batch1/2: C2为第二段充电倍率)
             'newstructure' (batch3: C2为放电倍率, 充电为 C1→Q1 后 1C)
    """
    m = re.match(r'([\d.]+)C\((\d+)\%\) - ([\d.]+)C', pr)
    if m:
        c1, q1, c2 = float(m.group(1)), float(m.group(2)), float(m.group(3))
        return c1, q1, c2, ('newstructure' if 'newstructure' in pr else 'standard')
    # 容错: '4C(31%)-5' 结尾少 C
    m1 = re.match(r'([\d.]+)C\((\d+)\%\) - ([\d.]+)', pr)
    if m1:
        c1, q1, c2 = float(m1.group(1)), float(m1.group(2)), float(m1.group(3))
        return c1, q1, c2, ('newstructure' if 'newstructure' in pr else 'standard')
    # 截断形式: '80%)-3.6C' -> C1=C2
    m2 = re.match(r'(\d+)\%\) - ([\d.]+)C', pr)
    if m2:
        q1 = float(m2.group(1)); c2 = float(m2.group(2))
        return c2, q1, c2, 'standard'
    return None, None, None, None

def clean_Qd(Qd, thr=0.20):
    """剔除异常循环：容量相对前值突变>thr 视为脏点"""
    Qd = np.array(Qd, dtype=float)
    out = np.where(Qd == 0, np.nan, Qd)
    for k in range(1, len(out)):
        if np.isnan(out[k]):
```

```

        continue
    prev = out[k-1]
    if not np.isnan(prev) and abs(out[k]-prev)/prev > thr:
        out[k] = np.nan
    return out

def main():
    os.makedirs(OUT_DIR, exist_ok=True)
    rows = []
    for fname in FILES:
        batch_id = fname.split('.')[0]
        with h5py.File(os.path.join(DATA_DIR, fname), 'r') as f:
            batch = f['batch']
            n = batch['cycle_life'].shape[0]
            for i in range(n):
                cell = f'{batch_id}_cell{i:02d}'
                cl_raw = float(np.array(f[batch['cycle_life']][i,0]).ravel()[0])
                cl = cl_raw if np.isfinite(cl_raw) else np.nan
                pr = ''.join(chr(int(x)) for x in
                    np.array(f[batch['policy_readable']][i,0]).ravel())
                c1, q1, c2, proto = parse_policy(pr, batch_id)
                summ = f[batch['summary']][i,0]
                Qd = clean_Qd(np.array(summ['QDischarge']).ravel())
                ct = np.array(summ['chargetime']).ravel()
                valid = Qd[1:][~np.isnan(Qd[1:])]
                Qinit = float(valid[0]) if len(valid) else np.nan
                soh_min = float(np.nanmin(Qd[1:]) / Qinit * 100) if len(valid) else
                    np.nan
                n_cycles = int(len(Qd) - 1)
                rows.append([cell, batch_id, pr, proto,
                    c1, q1, c2,
                    '' if np.isnan(cl) else round(cl),
                    n_cycles,
                    '' if np.isnan(Qinit) else round(Qinit, 4),
                    round(np.nanmean(ct[1:]), 3) if len(ct) > 1 else '',
                    round(float(ct[1]), 3) if len(ct) > 1 else '',
                    round(soh_min, 2)])
    path = os.path.join(OUT_DIR, 'battery_table.csv')
    with open(path, 'w', newline='', encoding='utf-8-sig') as fh:
        w = csv.writer(fh)
        w.writerow(['battery', 'batch', 'policy', 'protocol', 'C1_C', 'Q1_pct', 'C2_C',
            'cycle_life', 'n_cycles', 'Qinit_Ah', 'avg_chargetime_min', 'first_chargetime_min', 'mi
        w.writerows(rows)
    print(f'已输出 {len(rows)} 个电池 -> {path}')
    # 打印协议与缺失统计
    from collections import Counter
    print('协议分布:', Counter(r[3] for r in rows))

```

```

    print('cycle_life 缺失:', [r[0] for r in rows if r[7] == ''])

if __name__ == '__main__':
    main()

```

2.2 analysis1.py —— 问题一：数据整理与初步分析

```

# -*- coding: utf-8 -*-
"""B题 问题一：数据整理 + 寿命分布统计分析 + 长/短寿命策略识别"""
import pandas as pd, numpy as np, os, matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

BASE = r"C:\Users\one\Desktop\2026年校赛题目\B"
FIG = os.path.join(BASE, 'data_processed', 'figs')
os.makedirs(FIG, exist_ok=True)

df = pd.read_csv(os.path.join(BASE, 'data_processed', 'battery_table.csv'))
df['cycle_life'] = pd.to_numeric(df['cycle_life'], errors='coerce')
std = df[df['protocol'] == 'standard'].dropna(subset=['cycle_life']).copy()
new = df[df['protocol'] == 'newstructure'].dropna(subset=['cycle_life']).copy()

print("="*60)
print("一、数据整理结果")
print("="*60)
print(f"总电池数: {len(df)} | 有效寿命: {df['cycle_life'].notna().sum()} | 寿命缺失: {df['cycle_life'].isna().sum()}")
print(f"协议: standard(批次1+2) {len(std)} 个, newstructure(批次3) {len(new)} 个")
print(f"\n每批次寿命分布:")
for b in ['data_1', 'data_2', 'data_3']:
    s = df[df['batch']==b]['cycle_life'].dropna()
    print(f" {b}: n={len(s)} min={s.min():.0f} median={s.median():.0f} mean={s.mean():.0f} max={s.max():.0f}")

print("\n" + "="*60)
print("二、寿命总体分布")
print("="*60)
allc = df['cycle_life'].dropna()
print(f"全部有效电池: n={len(allc)} min={allc.min():.0f} Q1={allc.quantile(.25):.0f} median={allc.median():.0f} Q3={allc.quantile(.75):.0f} max={allc.max():.0f}")
print(f"寿命分位: p10={allc.quantile(.1):.0f} p25={allc.quantile(.25):.0f} p50={allc.quantile(.5):.0f} p75={allc.quantile(.75):.0f}")

```



```

p90={allc.quantile(.9):.0f}")

print("\n" + "="*60)
print("三、策略→寿命统计 (standard 批次, 每种策略的平均寿命)")
print("="*60)
grp = std.groupby('policy').agg(n=('cycle_life','size'),
    mean=('cycle_life','mean'),
                                std=('cycle_life','std'), med=('cycle_life','median'),
                                C1=('C1_C','mean'), Q1=('Q1_pct','mean'),
                                C2=('C2_C','mean'),
                                ct=('avg_chargetime_min','mean')).round(1)
grp = grp.sort_values('mean', ascending=False)
print(grp.to_string())

print("\n" + "="*60)
print("四、典型长寿命 vs 短寿命策略")
print("="*60)
top = grp.head(5); bot = grp.tail(5)
def show(tag, g):
    print(f"\n【{tag}】")
    for pol, row in g.iterrows():
        print(f" {pol:<22} 寿命均值={row['mean']:>6.0f}±{row['std']:>5.1f}
              (n={int(row['n'])}) "
              f"C1={row['C1']:.1f}C Q1={row['Q1']:.0f}% C2={row['C2']:.1f}C
              充电={row['ct']:.1f}min")
show("典型长寿命策略 (前5)", top)
show("典型短寿命策略 (后5)", bot)

long_ = grp.head(3); short_ = grp.tail(3)
print(f"\n长寿命均值: C1={long_['C1'].mean():.2f}C, Q1={long_['Q1'].mean():.0f}%,
      C2={long_['C2'].mean():.2f}C, 充电时间={long_['ct'].mean():.1f}min")
print(f"短寿命均值: C1={short_['C1'].mean():.2f}C, Q1={short_['Q1'].mean():.0f}%,
      C2={short_['C2'].mean():.2f}C, 充电时间={short_['ct'].mean():.1f}min")

print("\n" + "="*60)
print("五、cycle_life 与各参数的相关性 (standard)")
print("="*60)
for col, lab in [('C1_C','C1 第一阶段倍率'), ('Q1_pct','Q1 切换SOC'), ('C2_C','C2
    第二阶段倍率'), ('avg_chargetime_min','充电时间(min)')]:
    r = std[['cycle_life', col]].dropna().corr().iloc[0,1]
    print(f" cycle_life vs {lab:<14}: Pearson r = {r:.3f}")

print("\n" + "="*60)
print("六、newstructure 批次3 策略→寿命")
print("="*60)
g3 = new.groupby('policy').agg(n=('cycle_life','size'),
    mean=('cycle_life','mean'), C1=('C1_C','mean'),

```

```

    Q1=('Q1_pct', 'mean')).round(1).sort_values('mean', ascending=False)
print(g3.to_string())

# ===== 画图 =====
# 图1: 寿命分布直方图(按批次)
fig, axes = plt.subplots(1, 3, figsize=(14, 4), sharey=True)
for ax, b in zip(axes, ['data_1', 'data_2', 'data_3']):
    s = df[df['batch']==b]['cycle_life'].dropna()
    ax.hist(s, bins=20, color='steelblue', edgecolor='white')
    ax.set_title(f'{b} (n={len(s)})\nmedian={s.median():.0f}')
    ax.set_xlabel('cycle_life')
axes[0].set_ylabel('电池数')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig1_寿命分布_按批次.png'),
    dpi=120); plt.close()

# 图2: cycle_life vs 参数 (standard)
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, col, lab in zip(axes, ['C1_C', 'Q1_pct', 'C2_C', 'avg_chargetime_min'], ['C1
    (C)', 'Q1 (%)', 'C2 (C)', '充电时间 (min)']):
    ax.scatter(std[col], std['cycle_life'], s=30, alpha=.6)
    if col in ['C1_C', 'C2_C']:
        x = np.linspace(std[col].min(), std[col].max(), 100)
        z = np.polyfit(std[col], std['cycle_life'], 1)
        ax.plot(x, np.polyval(z, x), 'r--')
    ax.set_xlabel(lab); ax.set_ylabel('cycle_life')
fig.suptitle('cycle_life 与策略参数关系 (standard 批次1+2)')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig2_寿命vs参数.png'), dpi=120);
plt.close()

# 图3: 长短寿命策略 SOH 曲线对比
import h5py
def load_soh(fname, cell_idx):
    with h5py.File(os.path.join(r"C:\Users\one\Desktop", fname), 'r') as f:
        batch = f['batch']
        Qd = np.array(f[batch['summary']][cell_idx,0]['QDischarge']).ravel()
        return Qd[1:] / Qd[1] * 100

# 按策略名动态找索引 (data_1)
def find_cells(fname, policies):
    with h5py.File(os.path.join(r"C:\Users\one\Desktop", fname), 'r') as f:
        batch = f['batch']
        n = batch['cycle_life'].shape[0]
        found = {}
        for i in range(n):
            pr = ''.join(chr(int(x)) for x in
                np.array(f[batch['policy_readable']][i,0]).ravel())
            if pr in policies and pr not in found:

```

```

        found[pr] = i
    return found

LONG_POLS = ['4C(80%)-4C', '3.6C(80%)-3.6C', '8C(15%)-3.6C']
SHORT_POLS = ['5.4C(80%)-5.4C', '8C(35%)-3.6C', '8C(25%)-3.6C']
cell_idx = find_cells('data_1.mat', LONG_POLS + SHORT_POLS)
plt.figure(figsize=(9, 5))
for pol in LONG_POLS:
    soh = load_soh('data_1.mat', cell_idx[pol])
    plt.plot(np.arange(1, len(soh)+1), soh, linewidth=1.6, label=f'长寿 {pol}')
for pol in SHORT_POLS:
    soh = load_soh('data_1.mat', cell_idx[pol])
    plt.plot(np.arange(1, len(soh)+1), soh, '--', linewidth=1.6, label=f'短寿 {pol}')
plt.axhline(80, color='red', ls='--', alpha=.5); plt.text(5, 81, '80% SOH阈值',
    color='red')
plt.xlabel('循环数'); plt.ylabel('SOH (%)'); plt.legend(fontsize=8);
    plt.title('典型长/短寿命电池 SOH 衰减曲线 (批次1)')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig3_SOH曲线对比.png'), dpi=120);
    plt.close()

print(f"\n图表已保存到: {FIG}")
print(" fig1_寿命分布_按批次.png, fig2_寿命vs参数.png, fig3_SOH曲线对比.png")

```

2.3 analysis2.py ——问题二：量化模型

```

# -*- coding: utf-8 -*-
"""B题 问题二：充电策略参数与寿命衰减的定量关系模型
输出：回归系数/标准化系数/p值/重要性排序/交互分析 + 图4/图5
"""
import pandas as pd, numpy as np, os, math, sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy import stats

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

BASE = r"C:\Users\one\Desktop\2026年校赛题目\B"
FIG = os.path.join(BASE, 'data_processed', 'figs')
os.makedirs(FIG, exist_ok=True)

df = pd.read_csv(os.path.join(BASE, 'data_processed', 'battery_table.csv'))
df['cycle_life'] = pd.to_numeric(df['cycle_life'], errors='coerce')

```

```

std = df[df['protocol'] == 'standard'].dropna(subset=['cycle_life']).copy()
std['loglife'] = np.log10(std['cycle_life'])
std['T'] = std['avg_chargetime_min']
std['batch2'] = (std['batch'] == 'data_2').astype(float)
print(f"standard 电池数: {len(std)}")

# ----- 工具: 最小二乘回归 + 显著性 -----
def ols(X, y):
    X = np.asarray(X, float); y = np.asarray(y, float)
    n, p = X.shape
    Xd = np.column_stack([np.ones(n), X])
    coef, res, *_ = np.linalg.lstsq(Xd, y, rcond=None)
    yhat = Xd @ coef
    resid = y - yhat
    dof = n - (p + 1)
    sse = np.sum(resid ** 2)
    s2 = sse / dof
    XtX_inv = np.linalg.pinv(Xd.T @ Xd)
    se = np.sqrt(np.diag(XtX_inv) * s2)
    t = coef / se
    pv = 2 * (1 - stats.t.cdf(np.abs(t), dof))
    r2 = 1 - sse / np.sum((y - y.mean()) ** 2)
    r2adj = 1 - (1 - r2) * (n - 1) / dof
    return dict(coef=coef, se=se, t=t, pv=pv, r2=r2, r2adj=r2adj, n=n,
                rmse=math.sqrt(sse / n))

def show(name, R, varnames):
    print(f"\n### {name} (n={R['n']}, R^2={R['r2']:.4f}, adjR^2={R['r2adj']:.4f},
          RMSE={R['rmse']:.4f})")
    print(f"{'变量':<16}{'系数':>10}{'标准误':>10}{'t':>8}{'p值':>10}")
    for i, v in enumerate(varnames):
        print(f"{v:<16}{R['coef'][i+1]:>10.4f}{R['se'][i+1]:>10.4f}{R['t'][i+1]:>8.2f}{R['pv'][i+1]:>10.4f}")
    print(f"{'(截距)':<16}{R['coef'][0]:>10.4f}")

# ----- 1. 标准化回归(消除量纲, 含批次效应) -----
X_names = ['C1_C', 'Q1_pct', 'C2_C', 'T', 'batch2']
X = std[X_names].values
y = std['loglife'].values
Xs = (X - X.mean(axis=0)) / X.std(axis=0)
R = ols(Xs, y)
print("=" * 60); print("一、多变量回归 log10(cycle_life) ~ C1 + Q1 + C2 + 充电时间 +
      批次 (标准化)"); print("=" * 60)
show("标准化回归 (含批次效应)", R, X_names)
std_beta = dict(zip(X_names[:4], R['coef'][1:5]))
rank = sorted(std_beta.items(), key=lambda kv: -abs(kv[1]))
print("\n策略参数标准化系数排序(按绝对值, 批次效应已吸收):")
for v, b in rank:

```

```

    print(f" {v:<12} beta={b:+.3f}")
print(f"\n批次2效应: 系数={R['coef'][5]:+.3f}, p={R['pv'][5]:.4f}
      (负值表示批次2整体寿命偏低)")

# ----- 2. 原始尺度回归 (仅策略参数, 供论文引用系数) -----
Xr_names = ['C1_C', 'Q1_pct', 'C2_C', 'T']
Xr = std[Xr_names].values
Rr = ols(Xr, y)
show("原始尺度回归(仅策略参数)", Rr, Xr_names)

# ----- 3. Drop-one R2 相对重要性 (含批次) -----
print("\n" + "=" * 60); print("二、留一变量 R^2 变化 (相对重要性, 含批次效应)");
    print("=" * 60)
base_r2 = R['r2']
print(f"全模型 R^2 = {base_r2:.4f}")
imp = {}
for i, v in enumerate(X_names):
    Xd = np.delete(X, i, axis=1)
    r2i = ols(Xd, y)['r2']
    imp[v] = base_r2 - r2i
    print(f" 去掉 {v:<12}: R^2={r2i:.4f} ΔR^2={imp[v]:+.4f}")
print("重要性排序:", sorted(imp.items(), key=lambda kv: -kv[1]))

# ----- 4. 偏相关系数 -----
print("\n" + "=" * 60); print("三、偏相关系数 (控制其他变量后, 含批次)"); print("=" * 60)
def pcorr(X, y, idx):
    others = [j for j in range(X.shape[1]) if j != idx]
    Xo = np.column_stack([np.ones(len(y)), X[:, others]])
    c = np.linalg.lstsq(Xo, y, rcond=None)[0]
    ry = y - Xo @ c
    c2 = np.linalg.lstsq(Xo, X[:, idx], rcond=None)[0]
    rx = X[:, idx] - Xo @ c2
    r = np.corrcoef(rx, ry)[0, 1]
    return r
X_all = std[X_names].values
for i, v in enumerate(X_names):
    print(f" 偏相关 {v:<12}: r = {pcorr(X_all, y, i):+.3f}")

# ----- 5. 不同 SOC 区间高倍率影响的差异: 剂量模型 -----
print("\n" + "=" * 60); print("四、SOC 区间剂量模型 (验证不同区间高倍率影响差异)");
    print("=" * 60)
std['m1'] = std['C1_C'] * std['Q1_pct'] / 100 # 低SOC区高倍率电量剂量 (Ah@C1)
std['m2'] = std['C2_C'] * (80 - std['Q1_pct']) / 100 # 中高SOC区高倍率电量剂量 (Ah@C2)
X2 = std[['m1', 'm2']].values
R2m = ols(X2, std['loglife'].values)
show("log10(寿命) ~ m1(低SOC剂量) + m2(中高SOC剂量)", R2m, ['m1', 'm2'])
# 剂量标准化

```

```

X2s = (X2 - X2.mean(axis=0)) / X2.std(axis=0)
R2ms = ols(X2s, std['loglife'].values)
show("剂量模型(标准化)", R2ms, ['m1', 'm2'])
print("若 beta(m2) << beta(m1) 且 m2 显著为负, 说明中高SOC区高倍率充电更伤电池")

# ----- 6. 批次内一致性 -----
print("\n" + "=" * 60); print("五、分批次回归(稳健性, 批次内用 C1/Q1/C2)"); print("=" *
60)
within_names = ['C1_C', 'Q1_pct', 'C2_C']
for b in ['data_1', 'data_2']:
    sub = std[std['batch'] == b]
    if len(sub) < 10:
        continue
    Xb = sub[within_names].values
    yb = sub['loglife'].values
    Xbs = (Xb - Xb.mean(axis=0)) / Xb.std(axis=0)
    Rb = ols(Xbs, yb)
    show(f"批次 {b} (n={len(sub)}, 标准化)", Rb, within_names)
    print(" 批次内 C2 标准化系数:", f"{Rb['coef'][3]:+.3f}")

# ----- 7. 交互项 -----
print("\n" + "=" * 60); print("六、交互项检验"); print("=" * 60)
std['C1xQ1'] = std['C1_C'] * std['Q1_pct']
X3 = std[['C1_C', 'Q1_pct', 'C2_C', 'T', 'C1xQ1']].values
R3 = ols(X3, std['loglife'].values)
show("加 C1xQ1 交互", R3, ['C1', 'Q1', 'C2', 'T', 'C1xQ1'])

std['C2xQ1'] = std['C2_C'] * std['Q1_pct']
X4 = std[['C1_C', 'Q1_pct', 'C2_C', 'T', 'C1xQ1', 'C2xQ1']].values
R4 = ols(X4, std['loglife'].values)
show("加 C1xQ1 与 C2xQ1 交互", R4, ['C1', 'Q1', 'C2', 'T', 'C1xQ1', 'C2xQ1'])

# ----- 7.5 剂量+批次联合模型 -----
print("\n" + "=" * 60); print("六b、剂量+批次联合模型 (log10(寿命) ~ m1 + m2 +
batch2)"); print("=" * 60)
Xdm = std[['m1', 'm2', 'batch2']].values
Xdms = (Xdm - Xdm.mean(axis=0)) / Xdm.std(axis=0)
Rdm = ols(Xdms, std['loglife'].values)
show("剂量+批次联合模型(标准化)", Rdm, ['m1(低SOC剂量)', 'm2(中高SOC剂量)', 'batch2'])
print(f" m2 与 m1 标准化系数比值: {Rdm['coef'][2]/Rdm['coef'][1]:.2f}")

# ----- 8. 输出参数供论文引用 -----
res = dict(
    beta=std_beta, r2_full=R['r2'], imp=imp,
    m2_beta=R2ms['coef'][2], m1_beta=R2ms['coef'][1],
    m2_beta_joint=Rdm['coef'][2], m1_beta_joint=Rdm['coef'][1],
    r2_m2=R2ms['r2'], m2_pv=R2ms['pv'][2],

```

```

R_full=dict(R), Rr=dict(Rr), Rdm=dict(Rdm),
within1=dict(ols((std[std['batch']=='data_1'][['C1_C', 'Q1_pct', 'C2_C']].values -
std[std['batch']=='data_1'][['C1_C', 'Q1_pct', 'C2_C']].values.mean(0))
/
std[std['batch']=='data_1'][['C1_C', 'Q1_pct', 'C2_C']].values.std(0),
std[std['batch']=='data_1']['loglife'].values)),
batch_eff=R['coef'][5], batch_eff_pv=R['pv'][5],
)
np.save(os.path.join(BASE, 'data_processed', 'q2_results.npy'), res,
allow_pickle=True)

# ===== 图4: 各变量偏回归图 (控制其他参数与批次后) =====
fig, axes = plt.subplots(1, 4, figsize=(17, 4))
for ax, v in zip(axes, Xr_names):
    i = Xr_names.index(v)
    # 控制其他策略参数 + 批次后的偏回归
    others = [j for j in range(4) if j != i]
    other_cols = [Xr_names[j] for j in others] + ['batch2']
    Xo = np.column_stack([np.ones(len(y)), std[other_cols].values])
    ry = y - Xo @ np.linalg.lstsq(Xo, y, rcond=None)[0]
    rx = Xr[:, i] - Xo @ np.linalg.lstsq(Xo, Xr[:, i], rcond=None)[0]
    ax.scatter(rx, ry, s=30, alpha=.6, c='steelblue')
    z = np.polyfit(rx, ry, 1)
    xx = np.linspace(rx.min(), rx.max(), 50)
    ax.plot(xx, np.polyval(z, xx), 'r--', lw=1.2)
    ax.set_xlabel(f'{v} (残差化)', fontsize=10)
    ax.set_ylabel('log10(寿命) 残差', fontsize=10)
    # 偏相关基于4参数模型
    pr = pcorr(Xr, y, i)
    ax.set_title(f'{v} 偏相关={pr:.2f}', fontsize=10)
fig.suptitle('偏回归图: 控制其他变量与批次后, 各参数与寿命的关系 (standard, n=94)',
    fontsize=11)
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig4_偏回归图.png'), dpi=120);
plt.close()

# ===== 图5: 预测寿命热力图 =====
# 用 C1,Q1,C2 全模型在网格上预测 loglife (充电时间用均值)
Tmean = std['T'].mean()
grid_c1 = np.linspace(1, 8, 60)
grid_q1 = np.linspace(5, 80, 60)
# 固定 C2 = 3.6 与 6.0 两张子图, 展示 Q1×C1 面上的寿命
b = Rr['coef']
C1s, Q1s, C2s, Ts = X.mean(axis=0), X.std(axis=0), X.std(axis=0), X.std(axis=0)
def pred(c1, q1, c2):
    return 10 ** (b[0] + b[1]*c1 + b[2]*q1 + b[3]*c2 + b[4]*Tmean)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
for ax, c2 in zip(axes, [3.6, 6.0]):

```

```

Z = np.array([pred(c1, q1, c2) for q1 in grid_q1 for c1 in grid_c1])
im = ax.contourf(grid_c1, grid_q1, Z.T, levels=20, cmap='viridis_r')
ax.set_xlabel('C1 (C)'); ax.set_ylabel('Q1 (%)'); ax.set_title(f'C2 = {c2}C
    预测寿命')
cb = fig.colorbar(im, ax=ax)
cb.set_label('预测循环寿命')
# 标记实验覆盖点
for _, row in std[std['C2_C'] == c2].iterrows():
    ax.plot(row['C1_C'], row['Q1_pct'], 'r.', markersize=5)
fig.suptitle('策略参数空间上的寿命预测 (模型: log10(寿命) ~ C1+Q1+C2+T)', fontsize=11)
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig5_寿命热力图.png'), dpi=120);
plt.close()

print(f"\n图表: fig4_偏回归图.png, fig5_寿命热力图.png")

```

2.4 analysis3.py ——问题三：寿命预测模型

```

# -*- coding: utf-8 -*-
"""B题 问题三：基于早期循环数据的寿命预测模型
特征：早期SOH斜率/波动/潜伏时间/充电时间变化 + 策略参数
模型：Ridge / RandomForest / GradientBoosting, 5折重复交叉验证
评估：不同早期窗口长度 k 的预测误差
"""

import pandas as pd, numpy as np, os, sys, io, warnings
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
warnings.filterwarnings('ignore')
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.model_selection import RepeatedKFold
from sklearn.linear_model import Ridge, LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.preprocessing import StandardScaler

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

BASE = r"C:\Users\one\Desktop\2026年校赛题目\B"
FIG = os.path.join(BASE, 'data_processed', 'figs')
os.makedirs(FIG, exist_ok=True)

cy = pd.read_csv(os.path.join(BASE, 'data_processed', '每循环明细表.csv'))
bt = pd.read_csv(os.path.join(BASE, 'data_processed', 'battery_table.csv'))

```



```

bt['cycle_life'] = pd.to_numeric(bt['cycle_life'], errors='coerce')
bt = bt.dropna(subset=['cycle_life']).reset_index(drop=True)
# 充电时间异常清洗 (剔除 >40 min 的脏点)
cy = cy[cy['chargetime_h'] < 40].copy()

# ===== 特征提取 =====
WINDOWS = [5, 10, 20, 50, 100]
THRESH = [99.9, 99.8, 99.7, 99.5, 99.3, 99.0, 98.5, 98.0, 97.0, 96.0]

def extract_features(batt, k):
    d = cy[cy['battery'] == batt].sort_values('cycle').head(k)
    if len(d) < max(3, min(k, 5)):
        return None
    soh = d['SOH_pct'].values.astype(float)
    qd = d['Qd_Ah'].values.astype(float)
    ct = d['chargetime_h'].values.astype(float)
    x = np.arange(1, len(soh) + 1)
    # 线性拟合
    slope, intercept, r, p, se = stats.linregress(x, soh)
    resid = soh - (intercept + slope * x)
    cslope, cint, *_ = stats.linregress(x, ct)
    ctdiff = np.diff(ct)
    feats = {
        'soh_k': soh[-1], 'loss_k': 100 - soh[-1],
        'slope': slope, 'slope_r2': r ** 2,
        'soh_resid_std': np.std(resid), 'soh_firstdiff_std': np.std(np.diff(soh)),
        'qd_std': np.std(qd), 'qd_firstdiff_std': np.std(np.diff(qd)),
        'qd_slope': stats.linregress(x, qd).slope,
        'ct_first': ct[0], 'ct_k': ct[-1], 'ct_slope': cslope,
        'ct_diff_std': np.std(ctdiff), 'ct_drift': ct[-1] - ct[0],
    }
    # 潜伏时间: SOH 首次低于阈值的循环数 (窗口内未达到则记 k+0.5)
    for t in THRESH:
        below = d[d['SOH_pct'] < t]
        feats[f'lat_{t}'] = below['cycle'].min() if len(below) else (k + 0.5)
    return feats

# 提取所有电池特征
recs = []
for batt in bt['battery']:
    for k in WINDOWS:
        f = extract_features(batt, k)
        if f is None:
            continue
        f['battery'] = batt
        f['window'] = k
        recs.append(f)

```

```

F = pd.DataFrame(recs)
# 合并目标与策略参数
F = F.merge(bt[['battery', 'cycle_life', 'C1_C', 'Q1_pct', 'C2_C',
               'avg_chargetime_min', 'batch']],
            on='battery', how='left')
print(f"特征提取完成: {len(F)} 条 (电池x窗口)")

# ===== 建模评估 =====
DEG_FEATS = ['soh_k', 'loss_k', 'slope', 'slope_r2', 'soh_resid_std',
             'soh_firstdiff_std',
             'qd_std', 'qd_firstdiff_std', 'qd_slope',
             'ct_first', 'ct_k', 'ct_slope', 'ct_diff_std', 'ct_drift'] + [f'lat_{t}'
                                   for t in THRESH]
STRAT_FEATS = ['C1_C', 'Q1_pct', 'C2_C', 'avg_chargetime_min']
ALL_FEATS = DEG_FEATS + STRAT_FEATS

def make_model(name):
    if name == 'Ridge':
        return Ridge(alpha=1.0)
    if name == 'RandomForest':
        return RandomForestRegressor(n_estimators=300, min_samples_leaf=2,
                                     random_state=42, n_jobs=-1)
    return GradientBoostingRegressor(n_estimators=250, max_depth=2,
                                     learning_rate=0.05, random_state=42)

def cv_evaluate(feats_cols, label, model_name, k, seed=42):
    sub = F[(F['window'] == k) &
            (F[feats_cols].notna().all(axis=1))].reset_index(drop=True)
    X = sub[feats_cols].values
    yl = np.log10(sub['cycle_life'].values)
    rkf = RepeatedKFold(n_splits=5, n_repeats=8, random_state=seed)
    preds, trues = [], []
    for tr, te in rkf.split(X):
        m = make_model(model_name)
        sc = StandardScaler().fit(X[tr])
        Xtr, Xte = sc.transform(X[tr]), sc.transform(X[te])
        m.fit(Xtr, yl[tr])
        preds.append(m.predict(Xte))
        trues.append(yl[te])
    preds = np.concatenate(preds); trues = np.concatenate(trues)
    life_p = 10 ** preds; life_t = 10 ** trues
    mape = np.mean(np.abs(life_p - life_t) / life_t) * 100
    r2 = r2_score(trues, preds)
    return dict(mape=mape, r2=r2)

print("\n" + "=" * 60)
print("一、不同早期窗口长度下的预测误差 (5折x8次重复CV)")

```

```

print("=" * 60)
print(f"{'窗口k':>6}{'模型':<14}{'MAPE(%)':>10}{'R2(log10)':>12}")
summary = {}
for k in WINDOWS:
    best = None
    for mn in ['Ridge', 'RandomForest', 'GradientBoosting']:
        r = cv_evaluate(ALL_FEATS, 'all', mn, k)
        summary[(k, mn)] = r
        print(f"{'k':>6}{'mn':<14}{'mape':>10.1f}{'r2':>12.3f}")
        if best is None or r['mape'] < best[1]:
            best = (mn, r['mape'], r['r2'])
    print(f"    最优: {best[0]} MAPE={best[1]:.1f}% R2={best[2]:.3f}")
    print("-" * 50)

# ===== 特征组合消融 (k=100) =====
print("\n" + "=" * 60)
print("二、特征组合消融 (k=100, GradientBoosting)")
print("=" * 60)
for label, feats in [('仅早期衰减特征', DEG_FEATS),
                     ('仅策略参数', STRAT_FEATS),
                     ('早期特征+策略参数', ALL_FEATS)]:
    r = cv_evaluate(feats, label, 'GradientBoosting', 100)
    print(f" {label:<16}: MAPE={r['mape']:.1f}% R2={r['r2']:.3f}")

# ===== 最佳模型：拟合与特征重要性 (k=100) =====
sub = F[(F['window'] == 100) &
        (F[ALL_FEATS].notna().all(axis=1))].reset_index(drop=True)
X = sub[ALL_FEATS].values
yl = np.log10(sub['cycle_life'].values)
g = GradientBoostingRegressor(n_estimators=250, max_depth=2, learning_rate=0.05,
                              random_state=42)
g.fit(X, yl)
pred = g.predict(X)
imp = pd.Series(g.feature_importances_,
                index=ALL_FEATS).sort_values(ascending=False)
print("\n" + "=" * 60)
print("三、特征重要性 (GradientBoosting, k=100, 全特征拟合)")
print("=" * 60)
print(imp.head(15).to_string())
print(f"\n训练R2={r2_score(yl, pred):.3f}")

# 保存预测表 (对每个电池在k=100的留一/全拟合预测)
sub['pred_life'] = 10 ** pred
sub.to_csv(os.path.join(BASE, 'data_processed', 'q3_predictions.csv'),
           index=False, encoding='utf-8-sig')

# 保存结果
res = dict(summary={f'{'k'}_{'m'}': v for (k, m), v in summary.items()},

```

```

        imp=imp.to_dict(), feats=ALL_FEATS, windows=WINDOWS)
np.save(os.path.join(BASE, 'data_processed', 'q3_results.npy'), res,
        allow_pickle=True)

# ===== 图6: 预测 vs 实际 (k=100, GBM, 交叉验证预测) =====
# 用 RepeatedKfold 收集 OOF 预测
rkf = RepeatedKfold(n_splits=5, n_repeats=10, random_state=42)
oof = np.zeros(len(sub))
for tr, te in rkf.split(X):
    m = GradientBoostingRegressor(n_estimators=250, max_depth=2,
        learning_rate=0.05, random_state=42)
    m.fit(X[tr], yl[tr])
    oof[te] = m.predict(X[te])
life_t = 10 ** yl; life_p = 10 ** oof
fig, ax = plt.subplots(figsize=(7, 6.5))
ax.scatter(life_t, life_p, c=sub['batch'].map({'data_1': 'steelblue', 'data_2':
    'darkorange', 'data_3': 'green'}), s=35, alpha=.75)
lim = [100, 2200]
ax.plot(lim, lim, 'k--', lw=1)
ax.plot(lim, [x * 0.9 for x in lim], 'k:', lw=.8); ax.plot(lim, [x * 1.1 for x in
    lim], 'k:', lw=.8)
ax.set_xlim(lim); ax.set_ylim(lim); ax.set_xlabel('实际循环寿命');
    ax.set_ylabel('预测循环寿命')
mape_cv = np.mean(np.abs(life_p - life_t) / life_t) * 100
r2_cv = r2_score(yl, oof)
ax.set_title(f'早期100循环预测寿命 (GBM, 交叉验证)\nMAPE={mape_cv:.1f}%,
    R²={r2_cv:.3f}, n={len(sub)}')
from matplotlib.lines import Line2D
handles = [Line2D([0], [0], marker='o', color='none', markerfacecolor=c,
    markersize=8, label=b)
    for b, c in [('data_1', 'steelblue'), ('data_2', 'darkorange'), ('data_3',
        'green')]]
ax.legend(handles=handles, loc='upper left', fontsize=9)
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig6_寿命预测对比.png'),
    dpi=120); plt.close()

# ===== 图7: MAPE vs 早期窗口长度 =====
fig, ax = plt.subplots(figsize=(7.5, 5))
for mn, c, mk in [('Ridge', 'steelblue', 'o'), ('RandomForest', 'darkorange',
    's'), ('GradientBoosting', 'green', '^')]:
    vals = [summary[(k, mn)]['mape'] for k in WINDOWS]
    ax.plot(WINDOWS, vals, marker=mk, color=c, lw=1.8, label=mn)
ax.set_xlabel('早期循环窗口长度 k (个循环)'); ax.set_ylabel('寿命预测 MAPE (%)')
ax.set_xticks(WINDOWS); ax.grid(alpha=.3); ax.legend()
ax.set_title('预测误差随早期数据长度变化 (5折×8重复交叉验证)')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig7_MAPE_vs窗口.png'),
    dpi=120); plt.close()

```

```

# ===== 图8: 特征重要性 =====
fig, ax = plt.subplots(figsize=(7.5, 6))
imp_top = imp.head(12)[::-1]
ax.barh(imp_top.index, imp_top.values, color='steelblue')
ax.set_xlabel('特征重要性'); ax.set_title('寿命预测特征重要性 (k=100,
    GradientBoosting)')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig8_特征重要性.png'), dpi=120);
plt.close()

print(f"\n图表: fig6_寿命预测对比.png, fig7_MAPE_vs窗口.png, fig8_特征重要性.png")

# ===== 四、SOH 未来轨迹预测 (里程碑循环) =====
print("\n" + "=" * 60)
print("四、早期数据预测未来 SOH (k=20 特征, 里程碑循环 t=200/400/600/800/1000)")
print("=" * 60)
MILESTONES = [200, 400, 600, 800, 1000]
# 每个电池的 SOH@t (取最接近 t 的循环)
soh_at = {}
for batt in bt['battery']:
    d = cy[cy['battery'] == batt].sort_values('cycle')
    vals = {}
    for t in MILESTONES:
        near = d.iloc[(d['cycle'] - t).abs().argmin()]
        if abs(near['cycle'] - t) <= 5:
            vals[t] = near['SOH_pct']
        else:
            vals[t] = np.nan
    soh_at[batt] = vals
Fs = F[F['window'] == 20].reset_index(drop=True)
traj_res = {}
for t in MILESTONES:
    Fs[f'soh_{t}'] = Fs['battery'].map(lambda b: soh_at[b][t])
    tr = Fs.dropna(subset=[f'soh_{t}'] + ALL_FEATS).reset_index(drop=True)
    if len(tr) < 15:
        continue
    X = tr[ALL_FEATS].values
    y = tr[f'soh_{t}'].values
    rkf = RepeatedKFold(n_splits=5, n_repeats=8, random_state=7)
    oof = np.zeros(len(tr))
    for tt, te in rkf.split(X):
        m = GradientBoostingRegressor(n_estimators=200, max_depth=2,
            learning_rate=0.05, random_state=42)
        m.fit(X[tt], y[tt]); oof[te] = m.predict(X[te])
    mae = mean_absolute_error(y, oof)
    r2s = r2_score(y, oof)
    traj_res[t] = dict(mae=mae, r2=r2s, n=len(tr))

```

```

print(f" 里程碑 t={t}: R²={r2s:.3f} MAE={mae:.2f} pct (n={len(tr)})")

# ===== 图9: 预测SOH vs 实际SOH 轨迹 =====
# 挑选 3 个代表性电池 (长/中/短寿命), 展示早期20循环数据 + 里程碑预测 + 实际轨迹
sample = ['data_1_cell00', 'data_2_cell09', 'data_2_cell01'] # 长/中/短
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5), sharey=True)
for ax, batt in zip(axes, sample):
    d = cy[cy['battery'] == batt].sort_values('cycle')
    cl = bt[bt['battery'] == batt]['cycle_life'].values[0]
    ax.plot(d['cycle'], d['SOH_pct'], color='lightgray', lw=1.2, label='实际SOH轨迹')
    d20 = cy[cy['battery'] == batt].sort_values('cycle').head(20)
    ax.plot(d20['cycle'], d20['SOH_pct'], 'b-', lw=2, label='前20循环(训练用)')
    # 里程碑真实 SOH (预测目标)
    for t in MILESTONES:
        v = soh_at[batt][t]
        if np.isfinite(v):
            ax.plot(t, v, 'ro', ms=6)
    ax.axhline(80, color='red', ls='--', alpha=.5)
    ax.set_title(f'{batt}\ncycle_life={int(cl)}')
    ax.set_xlabel('循环数')
axes[0].set_ylabel('SOH (%)')
handles, labels = axes[0].get_legend_handles_labels()
axes[2].legend(handles, labels, fontsize=8, loc='lower left')
plt.suptitle('实际 SOH 轨迹与前 20 循环数据 (用于早期预测)', fontsize=11)
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig9_轨迹预测演示.png'),
    dpi=120); plt.close()

np.save(os.path.join(BASE, 'data_processed', 'q3_traj.npy'), traj_res,
    allow_pickle=True)
print("\n图9: fig9_轨迹预测演示.png")

```

2.5 analysis4.py ——问题四：策略优化

```

# -*- coding: utf-8 -*-
"""B题 问题四：兼顾充电时间与寿命衰减的快充策略优化
1) 充电时间模型 (物理理想模型 + 经验校准)
2) 寿命模型: SOC剂量模型  $\log_{10}(\text{寿命}) \sim m_1 + m_2$ ,  $m_1 = C_1 * Q_1 / 100$ ,  $m_2 = C_2 * (80 - Q_1) / 100$ 
3) 在观测参数水平构成的"合理邻域"内网格搜索, 提取 Pareto 前沿
4) 推荐策略 + 与典型长/短寿命策略对比
"""
import pandas as pd, numpy as np, os, sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

```

```

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

BASE = r"C:\Users\one\Desktop\2026年校赛题目\B"
FIG = os.path.join(BASE, 'data_processed', 'figs')
os.makedirs(FIG, exist_ok=True)

bt = pd.read_csv(os.path.join(BASE, 'data_processed', 'battery_table.csv'))
bt['cycle_life'] = pd.to_numeric(bt['cycle_life'], errors='coerce')
std = bt[bt['protocol'] == 'standard'].dropna(subset=['cycle_life']).copy()
std['loglife'] = np.log10(std['cycle_life'])
std['m1'] = std['C1_C'] * std['Q1_pct'] / 100
std['m2'] = std['C2_C'] * (80 - std['Q1_pct']) / 100
print(f"standard 电池: {len(std)}")

# ----- 1. 充电时间校准模型 -----
A = np.column_stack([std['Q1_pct']/100/std['C1_C'],
                    (80-std['Q1_pct'])/100/std['C2_C'], np.ones(len(std))])
yT = std['avg_chargetime_min'].values
w = np.linalg.lstsq(A, yT, rcond=None)[0]
Tpred = A @ w
T_r2 = 1 - np.sum((yT - Tpred)**2) / np.sum((yT - yT.mean())**2)
print(f"\n[1] 充电时间校准模型:  $T = \{w[0]:.2f\} \cdot (Q1/100)/C1 + \{w[1]:.2f\} \cdot ((80-Q1)/100)/C2 + \{w[2]:.2f\}$ ")
print(f" 校准  $R^2=\{T\_r2:.3f\}$ , MAE={np.mean(np.abs(yT-Tpred)):.2f} min (理想模型系数为60)")

def T80(C1, Q1, C2):
    return w[0]*(Q1/100)/C1 + w[1]*((80-Q1)/100)/C2 + w[2]

# ----- 2. 剂量寿命模型 -----
print("\n[2] 剂量寿命模型  $\log_{10}(\text{寿命}) \sim m1 + m2$ ")
life_res = {}
for tag, s in [('全部(standard)', std), ('批次data_1', std[std['batch']=='data_1']),
              ('批次data_2', std[std['batch']=='data_2'])]:
    A2 = np.column_stack([np.ones(len(s)), s['m1'], s['m2']])
    b, *_ = np.linalg.lstsq(A2, s['loglife'], rcond=None)
    p = A2 @ b
    r2 = 1 - np.sum((s['loglife']-p)**2)/np.sum((s['loglife']-s['loglife'].mean())**2)
    life_res[tag] = dict(b=b, r2=r2, n=len(s))
    print(f" {tag}<14: 截距={b[0]:.3f}, m1={b[1]:.4f}, m2={b[2]:.4f},  $R^2=\{r2:.3f\}$ , n={len(s)}")

# 优化用: 全部 standard 的剂量模型 (一般性设计建议)
b_all = life_res['全部(standard)']['b']

```

```

def life_model(C1, Q1, C2):
    m1 = C1*Q1/100; m2 = C2*(80-Q1)/100
    return 10 ** (b_all[0] + b_all[1]*m1 + b_all[2]*m2)

# ----- 3. 网格搜索 (观测水平合理邻域 + 剂量覆盖约束) -----
C1_lv = np.unique(std['C1_C'])
Q1_lv = np.unique(std['Q1_pct'])
C2_lv = np.unique(std['C2_C'])
C1m, Q1m, C2m = np.meshgrid(C1_lv, Q1_lv, C2_lv, indexing='ij')
m1_all = C1m*Q1m/100
m2_all = C2m*(80-Q1m)/100
T_all = T80(C1m, Q1m, C2m)
L_all = life_model(C1m, Q1m, C2m)

# 剂量覆盖约束: 候选设计的 (m1,m2) 必须落在实测剂量的凸包范围附近, 排除 8C(80%) 等未验证角点
m1_obs = std['m1'].values; m2_obs = std['m2'].values
m1_lo, m1_hi = m1_obs.min(), m1_obs.max()
m2_lo, m2_hi = m2_obs.min(), m2_obs.max()
feasible = (m1_all >= m1_lo) & (m1_all <= m1_hi) & (m2_all >= m2_lo) & (m2_all <=
    m2_hi)
n_all = C1m.size
n_feas = feasible.sum()
print(f"\n[3] 网格规模: {len(C1_lv)}x{len(Q1_lv)}x{len(C2_lv)} = {n_all} 个候选")
print(f" 剂量约束 (m1∈[{m1_lo:.2f},{m1_hi:.2f}], m2∈[{m2_lo:.2f},{m2_hi:.2f}])
    后可行: {n_feas} 个")

# Pareto 非支配点 (暴力筛选: 最大化寿命, 最小化充电时间)
Tf = T_all[feasible]; Lf = L_all[feasible]
C1f = C1m[feasible]; Q1f = Q1m[feasible]; C2f = C2m[feasible]
keep = np.ones(len(Tf), bool)
for i in range(len(Tf)):
    if not keep[i]:
        continue
    for j in range(len(Tf)):
        if i != j and Lf[j] >= Lf[i] and Tf[j] <= Tf[i] and (Lf[j] > Lf[i] or Tf[j]
            < Tf[i]):
            keep[i] = False
            break
Tf, Lf, C1f, Q1f, C2f = Tf[keep], Lf[keep], C1f[keep], Q1f[keep], C2f[keep]
P = pd.DataFrame(dict(C1=C1f, Q1=Q1f, C2=C2f, T=Tf,
    life=Lf)).drop_duplicates().sort_values('T')
print(f" Pareto 前沿点数: {len(P)}")

# 膝点: 归一化(寿命取对数)空间上离端点连线最远的点
def knee_point(P):
    TN = (P['T'] - P['T'].min())/(P['T'].max() - P['T'].min())
    L = np.log10(P['life'])

```



```

LN = (L - L.min())/(L.max() - L.min())
A = np.column_stack([TN.values, LN.values])
p1, p2 = A[0], A[-1]
v = p2 - p1; vn = np.linalg.norm(v)
# 点到端点连线距离 (2D 叉积)
d = np.abs(v[0]*(A[:,1]-p1[1]) - v[1]*(A[:,0]-p1[0])) / vn
return int(np.argmax(d)), d
kidx, darr = knee_point(P)
rec = P.iloc[kidx]
print(f" 膝点(推荐): C1={rec['C1']:.1f}C, Q1={rec['Q1']:.0f}%, C2={rec['C2']:.1f}C")
print(f"  T80={rec['T']:.2f} min, 预测寿命={rec['life']:.0f}")

# 备选: 充电时间预算约束下寿命最大 (更快的方案)
_fastsub = P.loc[P['T'] <= 12.7]
fast = _fastsub.loc[_fastsub['life'].idxmax()]
print(f" 备选(更快, T<=12.7min): C1={fast['C1']:.1f}C, Q1={fast['Q1']:.0f}%,
      C2={fast['C2']:.1f}C, "
      f"T80={fast['T']:.2f} min, 预测寿命={fast['life']:.0f}")
# 若退化为与膝点相同, 则固定为数据验证过的 4C(80%)-4C
if abs(fast['T'] - rec['T']) < 0.01:
    fast = dict(C1=4.0, Q1=80, C2=4.0, T=T80(4.0, 80, 4.0), life=life_model(4.0,
    80, 4.0))
    print(f" 备选(固定为 4C(80%)-4C): T80={fast['T']:.2f} min,
          预测寿命={fast['life']:.0f}")

# ----- 4. 对比 -----
fmt = lambda v: f'{v:g}' # 4.0 -> '4', 3.6 -> '3.6'
print("\n[4] 推荐策略与典型策略对比 (预测 + 实测)")
print(f"{'策略':<26}{'C1(C)':>6}{'Q1(%)':>6}{'C2(C)':>6}{'T80(min)':>9}{'预测寿命':>9}{'实测寿命':>9}")
rows = [
    ('推荐(膝点)', rec['C1'], rec['Q1'], rec['C2'], None),
    ('备选(更快)', fast['C1'], fast['Q1'], fast['C2'], None),
    ('长寿命 3.6C(80%)-3.6C', 3.6, 80, 3.6, None),
    ('长寿命 4C(80%)-4C', 4.0, 80, 4.0, None),
    ('长寿命 8C(15%)-3.6C', 8.0, 15, 3.6, None),
    ('6C(30%)-3.6C', 6.0, 30, 3.6, None),
    ('短寿命 1C(4%)-6C', 1.0, 4, 6.0, None),
    ('短寿命 2C(10%)-6C', 2.0, 10, 6.0, None),
]
for name, c1, q1, c2, _ in rows:
    T = T80(c1, q1, c2); L = life_model(c1, q1, c2)
    pol = f'{fmt(c1)}C({int(q1)}%)-{fmt(c2)}C'
    obs = std[std['policy'] == pol]
    obs_life = f"{obs['cycle_life'].mean():.0f}" if len(obs) else '-'
    print(f"{'name':<26}{'c1':>6.1f}{'q1':>6.0f}{'c2':>6.1f}{'T':>9.1f}{'L':>9.0f}{'obs_life':>9}")

# ----- 5. 与数据集中最优策略(按实测寿命/充电时间)对比 -----

```

```

print("\n[5] 数据集中 standard 各策略的平均实测寿命与平均充电时间")
pol_agg = std.groupby('policy').agg(n=('cycle_life','size'),
    life=('cycle_life','mean'),
    T=('avg_chargetime_min','mean')).reset_index()
best_life = pol_agg.sort_values('life', ascending=False).head(3)
print("实测寿命最高的3个策略:")
print(best_life.to_string(index=False))

# 保存
res = dict(w=w, T_r2=T_r2, b_all=b_all, life_res={k: {kk: (vv.tolist() if
    isinstance(vv, np.ndarray) else vv)
    for kk, vv in v.items()}} for k, v in
    life_res.items()),
    front=P[['C1','Q1','C2','T','life']].to_dict('records'),
    rec=rec.to_dict(), fast=fast.to_dict())
np.save(os.path.join(BASE, 'data_processed', 'q4_results.npy'), res,
    allow_pickle=True)

# ===== 图10: Pareto 前沿 =====
fig, ax = plt.subplots(figsize=(8, 6.5))
ax.scatter(T_all.ravel(), L_all.ravel(), s=4, alpha=.25, c='lightsteelblue',
    label='可行设计(观测水平邻域)')
ax.plot(P['T'], P['life'], 'o-', color='crimson', ms=6, lw=1.6, label='Pareto 前沿')
ax.plot(rec['T'], rec['life'], 'r*', ms=20, label=f'推荐策略(膝点):
    {rec["C1"]:.1f}C({rec["Q1"]:.0f}%) - {rec["C2"]:.1f}C')
ax.set_xlabel('充电时间 T80 (min)'); ax.set_ylabel('预测循环寿命')
ax.legend(fontsize=9, loc='upper right')
ax.set_title('快充策略 Pareto 前沿 (充电时间 vs 预测寿命)\n寿命模型: log10(寿命) ~
    低SOC剂量 + 中高SOC剂量')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig10_Pareto前沿.png'),
    dpi=120); plt.close()

# ===== 图11: 推荐充电策略电流剖面 =====
fig, ax = plt.subplots(figsize=(8, 4.6))
c1, q1, c2 = rec['C1'], rec['Q1'], rec['C2']
soc = np.array([0, q1, 80]); I = np.array([c1, c2, 1.0, 0.0])
ax.step(soc, I, where='post', lw=2.5, color='steelblue')
ax.axvline(q1, color='gray', ls=':', lw=1); ax.text(q1, c1+0.25, f'Q1={q1:.0f}%',
    ha='center', fontsize=9)
ax.text(q1/2, c1+0.2, f'{c1:.1f}C', ha='center', fontsize=10)
ax.text((q1+80)/2, c2+0.2, f'{c2:.1f}C', ha='center', fontsize=10)
ax.text(90, 1.1, '1C CC-CV', ha='center', fontsize=9)
ax.set_xlabel('SOC (%)'); ax.set_ylabel('充电倍率 (C)'); ax.set_xlim(0,100);
    ax.set_ylim(0, c1+1)
ax.set_title(f'推荐快充策略: {c1:.1f}C({q1:.0f}%) - {c2:.1f}C\nT80≈{T80(c1,q1,c2):.1f}
    min, 预测寿命≈{life_model(c1,q1,c2):.0f}')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig11_推荐策略剖面.png'),

```

```
    dpi=120); plt.close()

print(f"\n图表: fig10_Pareto前沿.png, fig11_推荐策略剖面.png")
```